

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO NORTE DE MINAS GERAIS - IFNMG - CAMPUS JANUÁRIA
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

KELTON MARTINS DIAS

ARQUITETURA PARA COORDENAÇÃO DE SISTEMAS ROBÓTICOS MULTIAGENTES NA INTERNET DAS COISAS ROBÓTICAS COM EDGE AI EMBARCADO

JANUÁRIA - MG
2026

KELTON MARTINS DIAS

ARQUITETURA PARA COORDENAÇÃO DE SISTEMAS ROBÓTICOS MULTIAGENTES NA INTERNET DAS COISAS ROBÓTICAS COM EDGE AI EMBARCADO

Trabalho de Conclusão de Curso apresentado como exigência parcial para obtenção do diploma do Curso de Bacharelado em Sistemas de Informação do Instituto Federal de Educação, Ciência e Tecnologia do Norte de Minas Gerais - IFNMG, Campus Januária.

Orientador: Prof. MSc. Felipe Augusto Oliveira Mota

JANUÁRIA - MG
2026

AGRADECIMENTOS

Agradeço, primeiramente, a Deus pelas bênçãos concedidas e por me proporcionar sabedoria, força e resiliência ao longo desta jornada. Aos meus pais, pela dedicação, pelo apoio incondicional e pelos esforços contínuos e árduos que tornaram possível a realização e a conquista desta importante etapa da minha vida. Ao meu orientador, Prof. MSc. Felipe Mota, pelo incentivo constante, pela orientação segura e pelas sugestões enriquecedoras, que contribuíram significativamente para a construção deste trabalho e para o meu desenvolvimento como estudante e pesquisador. Por fim, expresso minha imensa gratidão a uma pessoa especial que esteve presente em todos os momentos, oferecendo apoio, auxílio e confiança, cujo encorajamento foi fundamental para que eu acreditasse no meu potencial e me sentisse capaz de superar desafios e alcançar objetivos que antes pareciam distantes.

"A melhor maneira de prever o futuro é inventá-lo".

Alan Kay

RESUMO

Este trabalho propõe, implementa e avalia uma arquitetura de comunicação e colaboração baseada em mensageria orientada a eventos para coordenação de agentes robóticos heterogêneos em ambiente industrial simulado. O cenário foi modelado no CoppeliaSim EDU e integra dois manipuladores robóticos (Franka Emika Panda e UR10), um robô móvel (KUKA youBot) e um dispositivo embarcado ESP32-S3-WROOM-1-N16R8, coordenados pelo protocolo MQTT via broker Eclipse Mosquitto em uma arquitetura de quatro camadas. Três modelos de classificação de imagens por cor foram treinados, MobileNetV2 (95,18%), MobileNetV3 Small (93,57%) e CNN autoral (93,17%), e convertidos para o formato TensorFlow Lite com quantização inteira, resultando em reduções superiores a 89% no tamanho dos arquivos. A avaliação experimental da comunicação abrangeu 150 ciclos operacionais em três configurações de broker (local, Amazon EC2 e Intel Edison), com taxa de sucesso de 100% em todas as configurações, latência mediana inferior a 25 ms no ambiente local e superior a 200 ms nos ambientes externos. A implantação dos modelos no ESP32-S3 revelou discrepâncias significativas entre estimativas de consumo de memória realizadas fora do dispositivo e os valores reais exigidos pelo TensorFlow Lite Micro, aspecto raramente tratado na literatura. A CNN autoral foi adotada como modelo definitivo em razão do seu consumo reduzido de arena (38,4 KB na SRAM interna) e da ausência de erros de classificação no hardware real, ao contrário das arquiteturas de transferência de aprendizado. Com o modelo embarcado integrado ao fluxo operacional, o sistema realizou 200 operações completas de classificação em cinco execuções, com taxa de sucesso de 100% nos três ambientes e confiança média de 99,6%. Os resultados demonstram que arquiteturas IoRT capazes de integrar comunicação distribuída, coordenação multiagente e inferência embarcada são viáveis com hardware de baixo custo, e que a seleção de modelos para dispositivos com restrições severas de memória deve considerar o consumo real de arena medido diretamente no hardware alvo.

Palavras-chaves: Internet das Coisas Robóticas; MQTT; Sistemas Multiagentes; Edge AI; ESP32; TensorFlow Lite Micro; Visão Computacional.

ABSTRACT

This work proposes, implements, and evaluates an event-driven messaging architecture for communication and collaboration among heterogeneous robotic agents in a simulated industrial environment. The scenario was modeled in CoppeliaSim EDU and integrates two robotic manipulators (Franka Emika Panda and UR10), a mobile robot (KUKA youBot), and an embedded device (ESP32-S3-WROOM-1-N16R8), coordinated via the MQTT protocol using Eclipse Mosquitto as broker within a four-layer architecture. Three color-based image classification models were trained, namely MobileNetV2 (95.18%), MobileNetV3 Small (93.57%), and a custom CNN (93.17%), and converted to TensorFlow Lite format with integer quantization, achieving file size reductions exceeding 89%. The experimental evaluation of the communication layer covered 150 operational cycles across three broker configurations (local, Amazon EC2, and Intel Edison), achieving 100% success rate in all configurations, with median latency below 25 ms in the local environment and above 200 ms in external environments. Deployment of the models on the ESP32-S3 revealed significant discrepancies between memory consumption estimates made outside the device and the actual values required by TensorFlow Lite Micro, an aspect rarely addressed in the literature. The custom CNN was adopted as the definitive model due to its reduced arena consumption (38.4 KB in internal SRAM) and the absence of classification errors on real hardware, unlike the transfer learning architectures. With the embedded model integrated into the operational flow, the system performed 200 complete classification operations across five executions, achieving 100% success rate in all three environments and an average confidence of 99.6%. The results demonstrate that IoRT architectures capable of integrating distributed communication, multi-agent coordination, and embedded inference are feasible with low-cost hardware, and that model selection for memory-constrained devices must consider the actual arena consumption measured directly on the target hardware.

Keywords: Internet of Robotic Things; MQTT; Multi-Agent Systems; Edge AI; ESP32; TensorFlow Lite Micro; Computer Vision.

LISTA DE FIGURAS

Figura 1 – Arquitetura IoRT em múltiplos níveis de abstração. 16
Figura 2 – Fluxo de coordenação em sistemas multiagentes IoRT. 18
Figura 3 – Comunicação orientada a eventos com *broker* intermediário. 19
Figura 4 – Arquitetura do *middleware pub/sub* para sistemas multirrobô heterogêneos. . 21
Figura 5 – Arquitetura IoRT em camadas com processamento na borda. 23
Figura 6 – Taxonomia das técnicas de otimização de modelos para Edge AI. 25
Figura 7 – Comparativo de latência de publicação por plataforma robótica. 27
Figura 8 – Arquitetura do CoppeliaSim integrado a sistemas de controle externos. . . . 29
Figura 9 – Cenário de simulação CoppeliaSim, com identificação dos agentes 31
Figura 10 – Arquitetura IoRT multiagente de quatro camadas. 32
Figura 11 – Estrutura de tópicos MQTT do sistema capturada pelo MQTT Explorer. . . 33
Figura 12 – Exemplos de imagens do conjunto de dados nas três classes de classificação. 36
Figura 13 – Definição das arquiteturas dos modelos de classificação. 37
Figura 14 – Procedimento de ajuste fino em duas fases (MobileNetV2 e MobileNetV3 Small). 37
Figura 15 – Código de conversão dos modelos treinados para o formato *TensorFlow Lite*. 38
Figura 16 – Medição do consumo de arena de tensores com *TensorFlow Lite Micro*. . . . 39
Figura 17 – Consumo de recursos do broker Mosquitto nos ambientes AWS, Edison e local. 42
Figura 18 – Pacotes de dados por minuto nos ambientes AWS, Edison e local. 43
Figura 19 – Latência mediana por tópico MQTT nos ambientes local, AWS e Edison. . . 44
Figura 20 – Jitter por tópico MQTT nos ambientes local, AWS e Edison. 45
Figura 21 – Matrizes de confusão dos modelos avaliados. 46
Figura 22 – Curvas de acurácia durante o treinamento dos modelos. 47
Figura 23 – Curvas de erro durante o treinamento dos modelos. 47
Figura 24 – Comparação do tamanho dos modelos originais e das versões TensorFlow Lite. 48
Figura 25 – Relação entre tamanho do modelo TFLite e acurácia após conversão. 49
Figura 26 – Matrizes de confusão dos modelos convertidos para o formato TensorFlow Lite. 50
Figura 27 – Estimativa inicial do consumo de arena em relação ao limite prático do ESP32-S3. 51
Figura 28 – Sistema IoRT em operação: ESP32 físico, terminal de logs e cenário simulado. 53
Figura 29 – Estado da estante após execução 53
Figura 30 – Tempo de inferência no ESP32 para cada arquitetura. 54
Figura 31 – Latência mediana por tópico com inferência embarcada ativa nos ambientes. 54
Figura 32 – Jitter por tópico com inferência embarcada ativa nos ambientes local, AWS e Edison. 55

LISTA DE TABELAS

Tabela 1 – Hiperparâmetros de treinamento por arquitetura 35
Tabela 2 – Especificações do hardware embarcado utilizado na execução dos modelos . 40
Tabela 3 – Métricas de desempenho dos modelos de classificação no conjunto de teste . 46
Tabela 4 – Comparação entre modelos quanto ao uso de memória e acurácia 51
Tabela 5 – Comparação entre consumo de arena estimado no notebook e medido no ESP32 52

LISTA DE ABREVIATURAS E SIGLAS

6LoWPAN	IPv6 over Low-Power Wireless Personal Area Networks
AI	Artificial Intelligence (Inteligência Artificial)
API	Application Programming Interface (Interface de Programação de Aplicações)
AWS	Amazon Web Services
CNN	Convolutional Neural Network (Rede Neural Convolucional)
CoAP	Constrained Application Protocol
CPU	Central Processing Unit (Unidade Central de Processamento)
EC2	Elastic Compute Cloud
ESP-IDF	Espressif IoT Development Framework
GPU	Graphics Processing Unit (Unidade de Processamento Gráfico)
HTTP	Hypertext Transfer Protocol (Protocolo de Transferência de Hipertexto)
IEEE	Institute of Electrical and Electronics Engineers
IFNMG	Instituto Federal de Educação, Ciência e Tecnologia do Norte de Minas Gerais
IoRT	Internet of Robotic Things (Internet das Coisas Robóticas)
IoT	Internet of Things (Internet das Coisas)
IP	Internet Protocol (Protocolo de Internet)
LoRaWAN	Long Range Wide Area Network
MQTT	Message Queuing Telemetry Transport
NB-IoT	Narrowband Internet of Things
PSRAM	Pseudo-Static Random Access Memory (Memória Pseudo-Estática de Acesso Aleatório)
QoS	Quality of Service (Qualidade de Serviço)
RAM	Random Access Memory (Memória de Acesso Aleatório)

ReLU	Rectified Linear Unit (Unidade Linear Retificada)
REST	Representational State Transfer
RGB	Red, Green, Blue (Vermelho, Verde, Azul)
SDN	Software-Defined Networking (Redes Definidas por Software)
SMA	Sistemas Multiagentes
SRAM	Static Random Access Memory (Memória Estática de Acesso Aleatório)
TCP	Transmission Control Protocol (Protocolo de Controle de Transmissão)
TLS	Transport Layer Security (Segurança da Camada de Transporte)
UDP	User Datagram Protocol (Protocolo de Datagrama de Usuário)
Wi-Fi	Wireless Fidelity
XMPP	Extensible Messaging and Presence Protocol

SUMÁRIO

1	**INTRODUÇÃO**	**12**
1.1	Objetivos	13
1.1.1	Objetivo Geral	13
1.1.2	Objetivos Específicos	13
1.2	Justificativa	14
1.3	Organização do Trabalho	14
2	**REFERENCIAL TEÓRICO**	**15**
2.1	Internet das Coisas e Internet das Coisas Robóticas	15
2.2	Sistemas Multiagentes em Sistemas IoRT	17
2.3	Comunicação Distribuída e Coordenação Orientada a Eventos	18
2.4	Protocolos de Comunicação e Middleware em Sistemas IoRT	20
2.5	Arquiteturas Distribuídas em Sistemas IoRT	22
2.6	Edge AI e Execução de Redes Neurais em Dispositivos Embarcados	24
2.7	Avaliação de Desempenho em Sistemas IoRT	26
2.8	Ambientes de Simulação para Sistemas IoRT	28
3	**METODOLOGIA**	**30**
3.1	Construção do cenário experimental	30
3.2	Definição da arquitetura de software	31
3.3	Configuração do middleware MQTT	33
3.4	Avaliação experimental da comunicação	34
3.5	Treinamento dos modelos	35
3.6	Conversão e adaptação para execução embarcada	37
3.7	Embarque no ESP32 e verificação de viabilidade	38
3.8	Integração com o ambiente simulado e ciclo de decisão e atuação	40
4	**RESULTADOS**	**41**
4.1	Desempenho do Middleware MQTT	41
4.2	Treinamento perceptivo dos modelos	45
4.3	Impacto da Conversão e Quantização nos Modelos	48
4.4	Viabilidade de Embarque no ESP32	50
4.5	Resultados da Integração com Inferência Embarcada	52
5	**CONCLUSÃO**	**56**
5.1	Trabalhos Futuros	57

REFERÊNCIAS BIBLIOGRÁFICAS 58

APÊNDICES 61

APÊNDICE A – MÉTRICAS DE LATÊNCIA E JITTER POR TÓPICO MQTT — CENÁRIO SEM INFERÊNCIA EMBARCADA 62

APÊNDICE B – MÉTRICAS DE LATÊNCIA E JITTER POR TÓPICO MQTT — CENÁRIO COM INFERÊNCIA EMBARCADA 63

APÊNDICE C – TEMPO DE INFERÊNCIA NO ESP32-S3 POR ARQUITETURA DE MODELO 64

APÊNDICE D – DADOS BRUTOS DE LATÊNCIA E *JITTER* — CENÁRIO SEM INFERÊNCIA EMBARCADA 65

APÊNDICE E – DADOS BRUTOS DE LATÊNCIA E *JITTER* — CENÁRIO COM INFERÊNCIA EMBARCADA 66

APÊNDICE F – DIAGRAMA DE ATIVIDADES DO SISTEMA IORT PROPOSTO 67

1 INTRODUÇÃO

A robótica e os dispositivos inteligentes passaram a integrar ambientes industriais, hospitalares e urbanos nas últimas décadas, impulsionados pela convergência entre conectividade, inteligência artificial e avanços em automação (BATTH; NAYYAR; NAGPAL, 2018; VERMESAN *et al.*, 2020). Robôs deixaram de operar de forma isolada e passaram a atuar em ambientes dinâmicos e compartilhados, viabilizando aplicações que combinam mobilidade, percepção e autonomia decisória. Conforme Batth, Nayyar & Nagpal (2018) observa, os sistemas inteligentes passam a complementar e ampliar as capacidades humanas, redefinindo o papel da automação nos contextos produtivos e sociais.

À medida que robôs passam a operar em ambientes compartilhados, conectados e dinâmicos, surge a questão de como esses sistemas se comunicam, se organizam e colaboram para executar tarefas complexas. É nesse contexto que a Internet das Coisas (Internet of Things, IoT) assume papel estrutural, ao evoluir de uma abordagem centrada na conectividade de objetos para um ecossistema no qual dispositivos coletam, processam e compartilham dados, integrando-se a processos decisórios e ações físicas (SORRI; MUSTAFEE; SEPPÄNEN, 2022). No contexto industrial, essa integração constitui a base conceitual da Indústria 4.0, viabilizando a digitalização de processos produtivos e a comunicação entre máquinas, robôs e sistemas de gestão em tempo quase real (DALENOGARE *et al.*, 2018).

Com a consolidação da Indústria 4.0, tornaram-se visíveis limitações relacionadas à negligência de fatores humanos e à rigidez frente à incerteza (GHOBAKHLOO *et al.*, 2023). A Indústria 5.0 propõe superar essas restrições ao incorporar princípios de resiliência, sustentabilidade e colaboração entre humanos e sistemas inteligentes, nos quais robôs atuam como agentes adaptativos (GHOBAKHLOO *et al.*, 2023; BATTH; NAYYAR; NAGPAL, 2018).

A Internet das Coisas Robóticas (Internet of Robotic Things, IoRT) constitui o paradigma que integra robótica, IoT, computação em nuvem e inteligência artificial. Na perspectiva de Ray (2016), trata-se de um ecossistema no qual "dispositivos inteligentes podem monitorar eventos, fundir dados de sensores de diversas fontes, utilizar inteligência local e distribuída para determinar um curso de ação e, então, agir para controlar ou manipular objetos no mundo físico" (RAY, 2016, p. 2, tradução do autor). Diferente da IoT tradicional, de operação estática e específica, a IoRT incorpora robôs com mobilidade, capacidade de manipulação e autonomia decisória, ampliando o escopo das aplicações e a complexidade dos sistemas (VERMESAN *et al.*, 2020).

Essa complexidade expõe limitações técnicas concretas. Em aplicações como fábricas inteligentes, sistemas logísticos e robôs de serviço, a comunicação ocorre em ecossistemas distribuídos sujeitos a falhas de rede, latência variável e heterogeneidade tecnológica. Interoperabilidade entre protocolos, escalabilidade das arquiteturas comunicacionais e confiabilidade na troca de mensagens permanecem desafios sem solução consolidada, especialmente quando

múltiplos agentes precisam compartilhar estados e intenções de ação em tempo quase real (RAY, 2016; VERMESAN *et al.*, 2020).

A colaboração entre múltiplos agentes robóticos também apresenta lacunas relevantes. Muitos sistemas carecem de mecanismos explícitos para coordenação distribuída e resposta coletiva a eventos inesperados, com robôs operando de forma parcialmente isolada em cenários dinâmicos (ROMEO *et al.*, 2020). Essa limitação se torna crítica em fluxos de trabalho cooperativos, nos quais a falha de coordenação de um agente compromete o desempenho de toda a cadeia. A incorporação de inteligência artificial aprofunda o quadro: aplicações robóticas demandam respostas em tempo quase real, tornando inviável a dependência exclusiva de processamento centralizado em nuvem, enquanto a execução local enfrenta restrições severas de memória, energia e capacidade computacional (VERMESAN *et al.*, 2020).

Comunicação, colaboração e inteligência embarcada são tratadas, com frequência, como dimensões independentes tanto na literatura quanto nas implementações práticas. Essa separação dificulta a compreensão integrada de ecossistemas IoRT reais, nos quais falhas de comunicação afetam diretamente a coordenação e limitações computacionais repercutem sobre o comportamento coletivo dos agentes (SORRI; MUSTAFEE; SEPPÄNEN, 2022; BATTH; NAYYAR; NAGPAL, 2018). São poucos os estudos que demonstram, de forma aplicada e reproduzível, como articular essas três dimensões sob restrições reais de recursos, lacuna que fundamenta a investigação proposta.

Este trabalho propõe, implementa e avalia uma arquitetura de comunicação e colaboração baseada em mensageria orientada a eventos para coordenação de agentes robóticos heterogêneos em ambiente industrial simulado, com inferência de visão computacional executada localmente em dispositivo embarcado. O sistema integra manipuladores robóticos, robô móvel, dispositivo embarcado ESP32 e sensores virtuais, coordenados pelo protocolo MQTT via *broker* Eclipse Mosquitto.

1.1 OBJETIVOS

1.1.1 Objetivo Geral

Propor, implementar e validar uma arquitetura de comunicação e colaboração baseada em mensageria orientada a eventos para coordenação de agentes robóticos heterogêneos em ambiente industrial simulado, incorporando Edge AI para apoio à tomada de decisão distribuída.

1.1.2 Objetivos Específicos

1. Modelar um cenário industrial simulado com robôs manipuladores, robô móvel, sensores e dispositivo embarcado no CoppeliaSim;

2. Implementar uma arquitetura de comunicação em quatro camadas baseada no protocolo MQTT com *broker* Eclipse Mosquitto;

3. Desenvolver e adaptar modelos de redes neurais convolucionais para classificação de objetos por cores;
4. Executar e avaliar os modelos no dispositivo embarcado ESP32, verificando viabilidade em termos de memória e desempenho;
5. Integrar o módulo de inferência embarcada ao fluxo de coordenação distribuída via MQTT;
6. Avaliar a arquitetura de comunicação quanto a latência, *jitter* e taxa de sucesso em três configurações de *broker*;
7. Comparar o desempenho dos modelos antes e após a quantização inteira.

1.2 JUSTIFICATIVA

Este trabalho se justifica pela integração de elementos que, na literatura, costumam ser tratados de forma separada: protocolos de comunicação assíncrona baseados em eventos, coordenação de múltiplos agentes robóticos heterogêneos e inferência embarcada com restrições reais de recursos. Estudos sobre MQTT raramente abordam coordenação multiagente; trabalhos sobre sistemas multiagentes frequentemente tratam a comunicação como aspecto secundário; e pesquisas sobre Edge AI em dispositivos embarcados raramente integram a inferência a um fluxo distribuído operacional (SORRI; MUSTAFEE; SEPPÄNEN, 2022; BATTH; NAYYAR; NAGPAL, 2018). Tratar essas dimensões de forma conjunta, dentro de uma mesma arquitetura avaliada experimentalmente, é a principal contribuição deste trabalho.

A proposta combina ambiente simulado com componente físico real, o ESP32, permitindo observar o comportamento da arquitetura sem depender de infraestrutura robótica de alto custo (TAKAYA *et al.*, 2016). A avaliação experimental com métricas quantitativas confere ao trabalho caráter reproduzível, tornando-o referência aplicada para implementações futuras em sistemas IoRT. A escolha por tratar comunicação, coordenação e inteligência embarcada como dimensões interdependentes de uma mesma arquitetura é uma resposta direta ao tipo de fragmentação que a literatura aponta como obstáculo ao avanço do campo.

1.3 ORGANIZAÇÃO DO TRABALHO

O Capítulo 2 apresenta o referencial teórico sobre IoRT, sistemas multiagentes, comunicação distribuída, protocolo MQTT, arquiteturas em camadas, Edge AI e simulação. O Capítulo 3 descreve a construção do cenário, a arquitetura de *software*, a configuração do *middleware* e o desenvolvimento dos modelos embarcados. O Capítulo 4 apresenta os resultados de desempenho dos modelos, do *middleware* nas três configurações de *broker* e da integração com inferência embarcada. O Capítulo 5 sintetiza as contribuições, limitações e perspectivas para trabalhos futuros.

2 REFERENCIAL TEÓRICO

2.1 INTERNET DAS COISAS E INTERNET DAS COISAS ROBÓTICAS

A compreensão da IoRT começa pelo reconhecimento de que seus fundamentos conceituais transcendem a simples conectividade de dispositivos. A IoT, já apresentada na introdução, constitui a camada de infraestrutura sobre a qual a IoRT é construída: sensores, atuadores e dispositivos embarcados passam a integrar redes de comunicação capazes de coletar, transmitir e compartilhar dados de forma contínua. O que diferencia a IoRT desse substrato é a incorporação de sistemas robóticos como agentes ativos, com mobilidade, capacidade de manipulação e autonomia decisória, o que expande radicalmente o escopo das aplicações e, consequentemente, a complexidade arquitetural dos sistemas.

Ray (2016) define esse paradigma como a extensão da infraestrutura computacional para o mundo físico, onde dispositivos inteligentes percebem eventos, integram dados de múltiplas fontes e executam ações físicas como resultado de decisões locais ou distribuídas. Para o autor:

> A IoRT é um conceito no qual dispositivos inteligentes podem monitorar eventos, integrar dados de sensores de diversas fontes, utilizar inteligência local e distribuída para determinar um curso de ação e então agir para controlar ou manipular objetos no mundo físico (RAY, 2016, p. 1, tradução própria).

Essa definição é diretamente relevante para o contexto deste trabalho: o sistema proposto integra robôs manipuladores, um robô móvel, sensores e um dispositivo embarcado em um ambiente industrial simulado, no qual cada componente percebe eventos, processa informações e age sobre o ambiente físico de forma coordenada. A IoRT não é aqui apenas um referencial teórico distante, mas o paradigma no qual o sistema se insere e cujos desafios estruturais motivam as escolhas arquiteturais descritas na metodologia.

Essa natureza multidimensional se reflete diretamente na organização interna de seus sistemas. Na perspectiva de Afanasyev *et al.* (2019), a tecnologia IoRT pressupõe "a implementação de um sistema totalmente distribuído, que se estende por múltiplos níveis, abrangendo desde edifícios inteligentes, espaços inteligentes e robôs até sensores e atuadores"(AFANASYEV *et al.*, 2019, p. 3, tradução própria). Essa natureza multinível reforça a necessidade de uma organização arquitetural em camadas, exatamente o que fundamenta a seção de definição da arquitetura de *software* na metodologia: a separação entre Camada Física, Camada de Rede, Camada de Comunicação e Camada de Controle não é uma escolha arbitrária, mas o reflexo direto de como a literatura descreve a organização interna de sistemas IoRT.

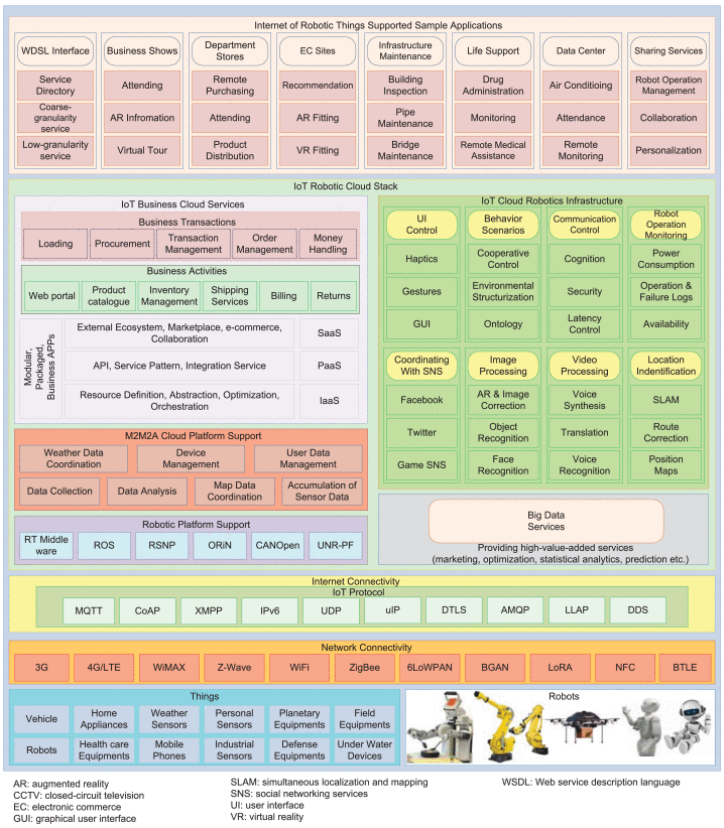
A integração de robôs em ecossistemas IoT, porém, não é isenta de dificuldades. Do ponto de vista técnico, Krejčí *et al.* (2025) sintetizam que esse processo "apresenta desafios significativos, incluindo a garantia de interoperabilidade, o processamento eficaz de dados e a escalabilidade do sistema"(KREJČÍ *et al.*, 2025, p. 20, tradução própria). Tais desafios, a

saber, a interoperabilidade entre agentes heterogêneos, o processamento eficiente de dados e a escalabilidade da arquitetura, correspondem diretamente às três dimensões investigadas neste trabalho: coordenação entre agentes de diferentes plataformas robóticas, inferência embarcada com recursos limitados e uma arquitetura de comunicação capaz de sustentar múltiplos agentes sem degradação. São esses desafios que motivam cada uma das escolhas tecnológicas descritas nas seções seguintes.

Villa *et al.* (2021) e Batth, Nayyar & Nagpal (2018) convergem ao destacar que a IoRT representa um nível qualitativamente superior ao da IoT convencional, ao incorporar percepção, computação, comunicação e controle como pilares interdependentes de uma mesma arquitetura. Essa interdependência é central para compreender por que falhas em qualquer um desses pilares comprometem o funcionamento do sistema como um todo, perspectiva que justifica a avaliação integrada da comunicação, da coordenação e da inferência embarcada realizada nesta pesquisa.

A Figura 1 ilustra esse ecossistema, mostrando como dispositivos físicos, redes de comunicação e serviços computacionais se organizam nos diferentes níveis de abstração de um sistema IoRT.

Figura 1 – Arquitetura IoRT em múltiplos níveis de abstração.



Fonte: Adaptado de Ray (2016).

2.2 SISTEMAS MULTIAGENTES EM SISTEMAS IORT

A modelagem de sistemas IoRT como estruturas multiagentes não é uma escolha de design entre outras, mas a forma mais natural de representar ambientes nos quais múltiplos dispositivos heterogêneos precisam cooperar sem controle centralizado. A abordagem multiagente considera cada componente do sistema, seja robô, sensor ou dispositivo embarcado, como uma entidade autônoma com percepção local, capacidade de processamento própria e capacidade de interagir com os demais agentes por meio de troca de mensagens. O comportamento global do sistema não é programado diretamente, mas emerge da cooperação entre esses agentes individuais.

Astudillo & Missael (2022) afirmam que essa abordagem permite organizar sistemas robóticos como estruturas compostas por múltiplos agentes cooperativos, nos quais o comportamento coletivo resulta diretamente da interação entre componentes individuais, cada um com suas capacidades e responsabilidades definidas. No sistema proposto neste trabalho, esse princípio se materializa na distribuição de funções entre os manipuladores Franka Emika Panda e UR10, o robô móvel KUKA youBot e o dispositivo embarcado ESP32: cada agente executa uma etapa específica do fluxo operacional, e a coordenação entre eles sustenta a execução do ciclo completo de coleta, transporte e armazenamento.

Essa concepção encontra respaldo na definição de sistemas multirrobô como caso particular dos sistemas multiagentes em IoRT: trata-se de "sistemas robóticos compostos por múltiplos agentes que operam em um mesmo ambiente"(AFANASYEV *et al.*, 2019, p. 4, tradução própria). O elemento que mais distingue arquiteturas multiagentes de abordagens centralizadas é justamente a forma como a coordenação entre os agentes é estabelecida. Em vez de depender de um controlador único que monitora e comanda todos os componentes, cada agente reage aos eventos que recebe, publica seus estados e decide suas ações com base nas informações disponíveis localmente.

Kabir, Tham & Chang (2023) descrevem esse modelo como predominante em sistemas robóticos IoRT com robôs móveis, caracterizando-o como altamente robusto por sua capacidade de continuar operando mesmo diante da falha de um dos membros. Os autores detalham:

> A abordagem de rede mais comum dos robôs móveis em IoRT é a descentralizada. Nessa abordagem, os robôs são governados com base em sua situação local, considerando diretrizes e objetivos específicos. É um método altamente robusto por sua capacidade de superar falhas de qualquer membro da equipe e por não centralizar o controle (KABIR; THAM; CHANG, 2023, p. 11, tradução própria).

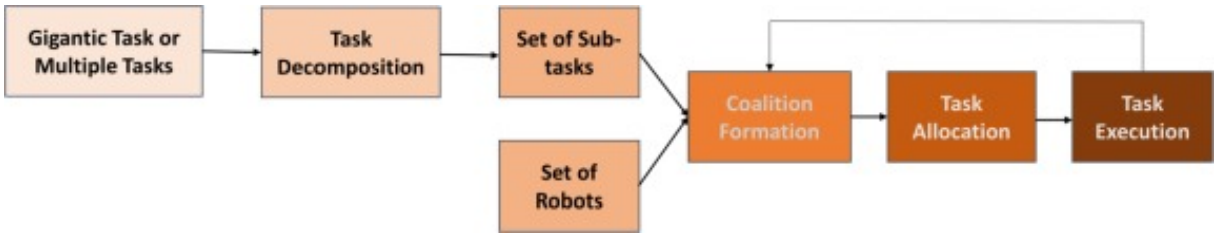
Essa descrição corresponde diretamente ao modelo adotado na metodologia: os *scripts* de controle dos manipuladores e do robô móvel operam de forma independente entre si, publicando estados e reagindo a eventos por meio de tópicos MQTT específicos. Não existe um módulo de controle centralizado que sequencie explicitamente todas as ações, pois a coordenação

emerge da troca de mensagens e da reação individual de cada agente aos eventos que recebe. A justificativa para essa escolha encontra apoio direto no referencial apresentado por Kabir et al.

Zegarra, Aranibar & Cardinale (2024) demonstram, na prática, como esse modelo pode ser implementado com robôs de plataformas distintas. Os autores mediram as latências de publicação em um sistema com cinco robôs de diferentes sistemas operacionais, obtendo valores que variavam de aproximadamente 31 ms a 166 ms, evidenciando que a coordenação multiagente em IoRT é tecnicamente viável, mas impõe requisitos de desempenho que precisam ser medidos e avaliados. Essa mesma perspectiva fundamenta a avaliação experimental da arquitetura de comunicação descrita na metodologia.

A Figura 2 apresenta o fluxo de coordenação típico em sistemas multiagentes aplicados à IoRT, ilustrando como tarefas são decompostas, agentes são agrupados e responsabilidades são distribuídas.

Figura 2 – Fluxo de coordenação em sistemas multiagentes IoRT.



Fonte: Adaptado de Kabir, Tham & Chang (2023).

2.3 COMUNICAÇÃO DISTRIBUÍDA E COORDENAÇÃO ORIENTADA A EVENTOS

Em sistemas nos quais múltiplos agentes físicos operam de forma simultânea e assíncrona, a comunicação não é um elemento periférico da arquitetura, mas o mecanismo que torna possível a coordenação. Cada agente possui apenas uma visão parcial do sistema: o manipulador Franka sabe o que ocorre em sua área de atuação, o robô móvel conhece sua posição e estado, e o dispositivo embarcado processa as imagens que recebe. O que permite que esses agentes coordenem suas ações sequenciais e interdependentes é o fluxo de mensagens que circula entre eles. Ray (2016) afirma que a conectividade transforma dispositivos físicos em componentes ativos de sistemas distribuídos, tornando a comunicação um elemento estrutural da arquitetura, não um detalhe de implementação.

Essa dependência estrutural tem implicações concretas sobre a confiabilidade do sistema. Conforme alertado por Afanasyev *et al.* (2019), "quando a conexão é interrompida, uma falha ocorre e pode inviabilizar o alcance dos objetivos pelos robôs"(AFANASYEV *et al.*, 2019, p. 6, tradução própria). Tal advertência justifica diretamente a avaliação da arquitetura de comunicação realizada na metodologia, na qual latência, *jitter* e taxa de sucesso são mensurados ao longo de múltiplos ciclos operacionais. O objetivo não é apenas verificar se a comunicação funciona,

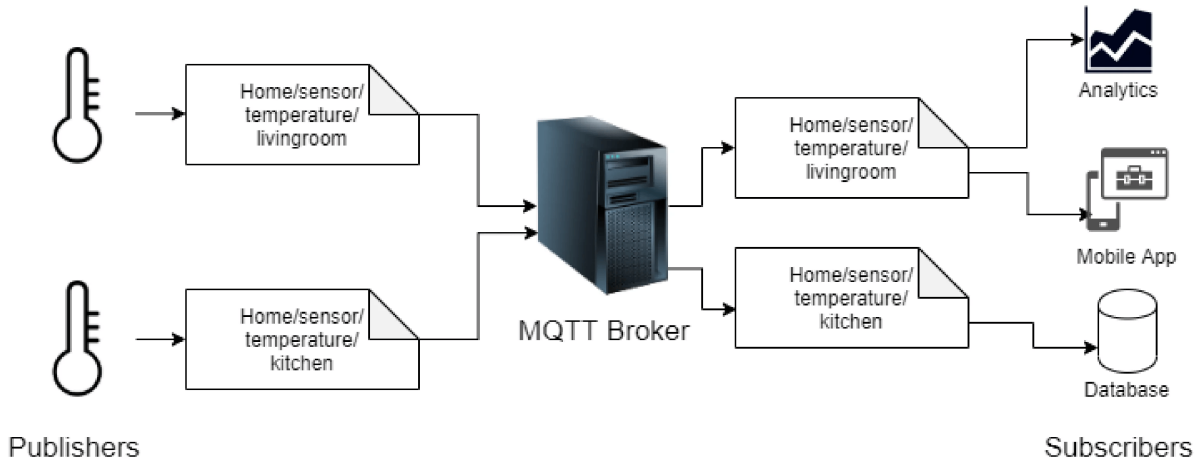
mas caracterizar seu comportamento temporal, pois é esse comportamento que determina se a coordenação entre os agentes pode ser sustentada em condições reais de operação.

Protocolos baseados em requisição e resposta, como o HTTP, introduzem acoplamento temporal entre os componentes: emissor e receptor precisam estar disponíveis ao mesmo tempo para que a comunicação ocorra. Em sistemas nos quais múltiplos eventos são gerados de forma contínua e assíncrona, esse modelo torna-se inadequado. Koutras *et al.* (2024) explicam que esse tipo de dependência aumenta o acoplamento entre os módulos e prejudica a escalabilidade do sistema. Como alternativa, modelos de comunicação orientados a eventos têm sido amplamente adotados em sistemas IoRT, pois permitem que os agentes publiquem eventos e reajam a notificações sem aguardar respostas síncronas.

Nesse modelo, os agentes publicam mensagens em canais de comunicação e outros agentes se inscrevem para recebê-las, sem interação direta e contínua entre eles. Mishra, Mishra & Kertesz (2021) afirmam que essa abordagem melhora a robustez do sistema, uma vez que a indisponibilidade temporária de um agente não impede o funcionamento dos demais componentes. Živić, Nemec & Bojović (2023) reforçam que o modelo assíncrono reduz o acoplamento e melhora a escalabilidade, dois requisitos diretamente relevantes para o sistema proposto, no qual novos agentes devem poder ser integrados sem alteração dos demais módulos. Essa escolha é o fundamento conceitual para a adoção do modelo de publicação e assinatura descrito na metodologia.

A Figura 3 representa esse modelo de comunicação, evidenciando o papel do componente intermediário na distribuição das mensagens entre os agentes.

Figura 3 – Comunicação orientada a eventos com *broker* intermediário.



Fonte: Adaptado de Mishra, Mishra & Kertesz (2021).

Villa *et al.* (2021) chamam atenção para o custo de latência gerado pelo processamento simultâneo de dados de múltiplos robôs, observando que situações de alta carga podem resultar em perda de mensagens e inconsistências operacionais. Essa advertência é especialmente

relevante para o experimento de avaliação da arquitetura de comunicação, no qual diferentes configurações de *broker* são testadas para identificar qual oferece o melhor equilíbrio entre latência e confiabilidade sob a carga gerada pelos ciclos operacionais. A caracterização das métricas de *jitter* e taxa de sucesso completa essa análise ao permitir observar não apenas o valor médio de latência, mas a variabilidade temporal e a estabilidade do sistema ao longo de múltiplas execuções.

2.4 PROTOCOLOS DE COMUNICAÇÃO E MIDDLEWARE EM SISTEMAS IORT

A escolha de um protocolo de comunicação em sistemas IoRT não é apenas uma decisão técnica: ela define a natureza das interações entre os agentes, o nível de acoplamento entre os módulos e os limites práticos da escalabilidade e confiabilidade do sistema. Vermesan *et al.* (2020) afirmam que a interoperabilidade entre dispositivos constitui um requisito fundamental para sistemas distribuídos, sendo viabilizada por protocolos que abstraem as particularidades do *hardware* e da infraestrutura de rede. No contexto desta pesquisa, essa escolha recaiu sobre o MQTT, e as razões para isso decorrem diretamente das características do protocolo descritas na literatura.

Villa *et al.* (2021) apresentam os protocolos mais utilizados em sistemas IoRT, organizados por camada de comunicação. Na camada de aplicação, destacam-se CoAP, MQTT, REST API e XMPP; na camada de transporte, UDP e TCP; na camada de rede, IP e 6LoWPAN; e na camada física, Wi-Fi, IEEE 802.15.4, Bluetooth, LoRaWAN e NB-IoT. Essa diversidade reflete a heterogeneidade dos ambientes IoRT e a necessidade de selecionar protocolos adequados às restrições de cada dispositivo e ao perfil de comunicação exigido pela aplicação. Entre os protocolos de aplicação listados, o MQTT destaca-se por apresentar o menor *overhead* e por ser nativamente orientado ao modelo de publicação e assinatura, tornando-o especialmente adequado para sistemas nos quais múltiplos agentes trocam eventos de forma assíncrona.

Mishra, Mishra & Kertesz (2021) descrevem o protocolo como:

> O MQTT é um protocolo de mensagens de publicação e assinatura simples, leve, baseado em TCP/IP. O cabeçalho binário torna o MQTT um protocolo leve para transmissão de dados de telemetria entre dispositivos com recursos restritos (MISHRA; MISHRA; KERTESZ, 2021, p. 3, tradução própria).

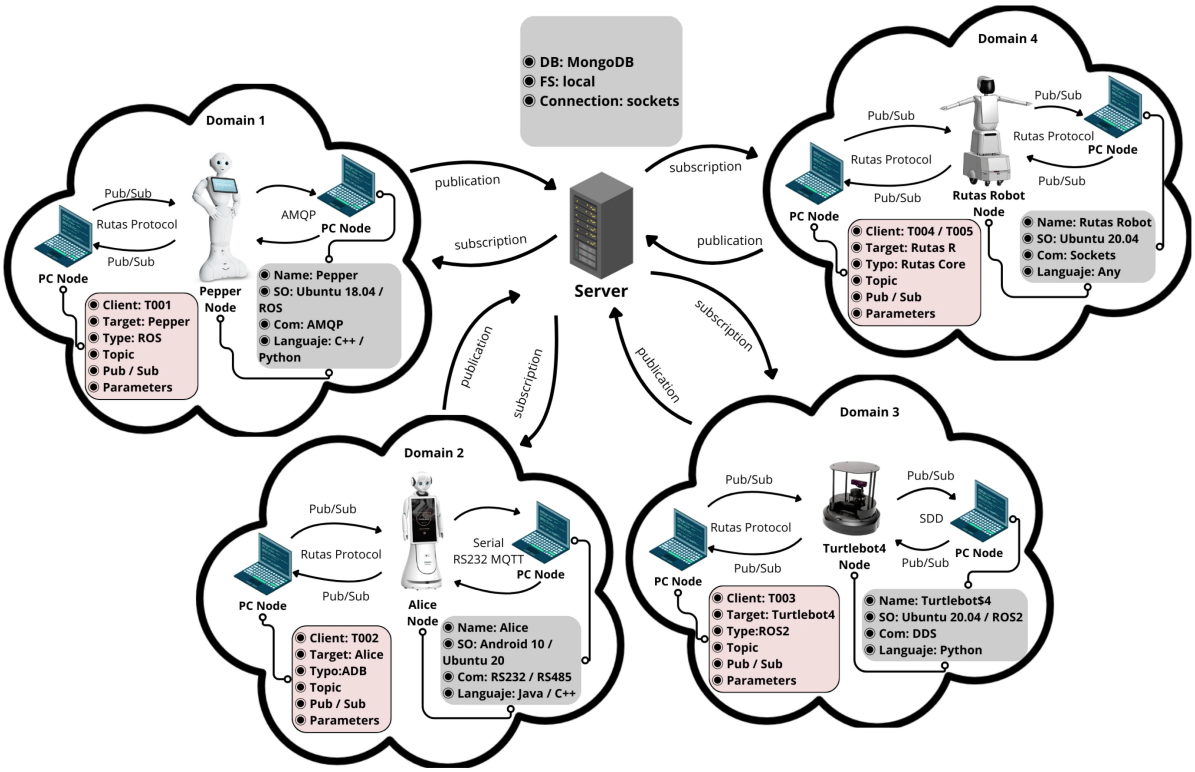
Essa leveza é especialmente relevante no contexto do sistema proposto, que inclui um dispositivo embarcado com recursos computacionais limitados, o ESP32. Em dispositivos com restrições de processamento e memória, o *overhead* do protocolo de comunicação impacta diretamente o desempenho da infraestrutura. O MQTT minimiza esse custo por meio de um cabeçalho binário compacto, tornando-o compatível com dispositivos que não poderiam sustentar protocolos mais pesados. Essa característica é um dos fundamentos técnicos que justificam a adoção do MQTT na arquitetura proposta.

O MQTT oferece três níveis de Qualidade de Serviço (QoS), que definem o comportamento de entrega das mensagens e permitem equilibrar confiabilidade e desempenho conforme

os requisitos da aplicação. O QoS 0 opera no modo *fire and forget*, entregando a mensagem no máximo uma vez, sem confirmação. O QoS 1 garante entrega de pelo menos uma vez, com potencial de duplicatas. O QoS 2 assegura entrega exatamente uma vez por meio de um *handshake* de quatro etapas, sendo a mais confiável, mas com maior custo de latência. Mishra, Mishra & Kertesz (2021) demonstram que esses mecanismos permitem adaptar o protocolo a perfis operacionais distintos, e os dados obtidos em seu estudo revelam o impacto dessa escolha: o *broker* EMQX manteve latência média de 0,87 ms com QoS 1 em ambiente com 1.000 clientes simultâneos, enquanto o HiveMQ atingiu 58,48 ms sob as mesmas condições, diferença de mais de 60 vezes. Esses resultados destacam a importância de avaliar empiricamente o desempenho do *broker* sob a carga específica do sistema, exatamente o que a metodologia propõe ao comparar diferentes configurações do Eclipse Mosquitto.

A **??** representa a arquitetura de *middleware* adotada em um sistema IoRT multirrobô heterogêneo, ilustrando como um servidor central intermedia a comunicação entre domínios robóticos distintos por meio do modelo de publicação e assinatura.

Figura 4 – Arquitetura do *middleware pub/sub* para sistemas multirrobô heterogêneos.



Fonte: Adaptado de Zegarra, Aranibar & Cardinale (2024).

Além do protocolo em si, o *middleware* exerce papel igualmente relevante na organização da comunicação distribuída. Essa camada intermediária abstrai os detalhes da comunicação de baixo nível e fornece mecanismos para gerenciar estados distribuídos, coordenar interações

entre agentes heterogêneos e tratar falhas. Para sistemas multirrobô heterogêneos, a solução proposta por Zegarra, Aranibar & Cardinale (2024) consiste em "um *middleware* baseado em um modelo de comunicação de publicação e assinatura generalizado, adaptável aos sistemas robóticos envolvidos, com visão voltada à computação em nuvem"(ZEGARRA; ARANIBAR; CARDINALE, 2024, p. 7, tradução própria). Rahman & Hussain (2020) afirmam que essa camada permite estruturar a comunicação entre componentes heterogêneos, reduzindo a complexidade da arquitetura e facilitando a interoperabilidade. No sistema proposto, o Eclipse Mosquitto cumpre esse papel: ele centraliza a distribuição de mensagens entre os manipuladores, o robô móvel, os sensores e o dispositivo embarcado, mantendo os módulos desacoplados e possibilitando que cada agente opere de forma independente. A escolha do Mosquitto entre outros *brokers* disponíveis decorre de sua leveza, compatibilidade com a biblioteca paho-mqtt em Python e ampla adoção em sistemas IoT e IoRT, conforme evidenciado nos estudos de Mishra, Mishra & Kertesz (2021).

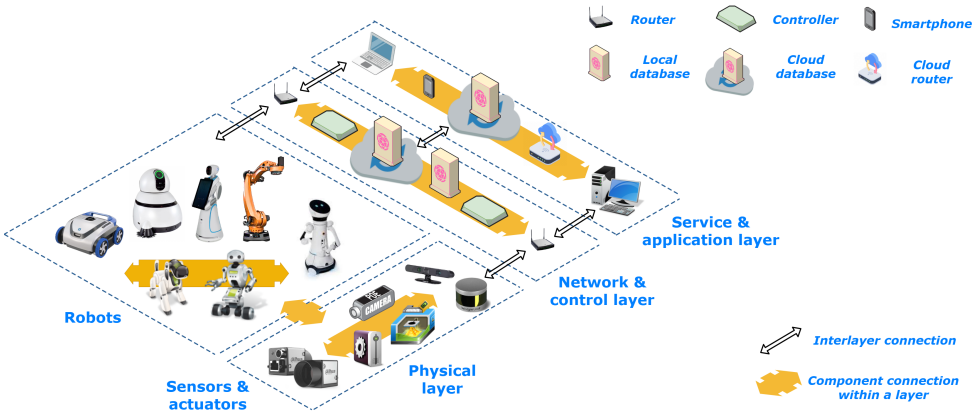
2.5 ARQUITETURAS DISTRIBUÍDAS EM SISTEMAS IORT

A organização arquitetural de sistemas IoRT determina a forma como seus componentes interagem, como as responsabilidades são distribuídas e como o sistema responde a falhas e variações operacionais. Ray (2016) afirma que a IoRT depende de uma arquitetura capaz de reunir percepção, comunicação e atuação em um único ecossistema operacional, e que essa reunião não ocorre espontaneamente: ela exige uma estrutura formal que organize o fluxo de dados e coordene as interações entre os componentes. Sem essa organização, a heterogeneidade de dispositivos típica de sistemas IoRT torna a integração inviável.

A abordagem mais consolidada na literatura consiste na organização do sistema em camadas funcionais com responsabilidades bem delimitadas. Kabir, Tham & Chang (2023) identificam uma estrutura de três camadas amplamente adotada em sistemas IoRT para robôs móveis: a camada física, responsável pelos dispositivos de percepção e atuação; a camada de rede, que viabiliza a conectividade entre os componentes; e a camada de serviços e controle, que engloba o processamento das informações e a tomada de decisão. Abughazala *et al.* (2026) explicam que essa organização modular permite modificar ou substituir componentes sem comprometer o funcionamento global do sistema, o que aumenta a manutenibilidade e facilita a evolução progressiva da arquitetura.

A Figura 5 ilustra essa estrutura, representando as três camadas e destacando a posição do processamento na borda no contexto da camada de controle. Essa representação é relevante para este trabalho porque evidencia com clareza onde o dispositivo embarcado ESP32 se insere na arquitetura: ele opera na fronteira entre a camada física e a camada de controle, executando a inferência localmente e transmitindo apenas o resultado classificado para os demais agentes do sistema via MQTT.

Figura 5 – Arquitetura IoRT em camadas com processamento na borda.



Fonte: Adaptado de Afanasyev *et al.* (2019).

A camada física concentra os dispositivos responsáveis pela percepção e atuação no ambiente, compreendendo robôs, sensores e dispositivos embarcados. Esses componentes constituem a interface entre o sistema computacional e o mundo físico, e são os que efetivamente executam as ações que o sistema é projetado para realizar. Villa *et al.* (2021) destacam que robôs e sensores desempenham papel fundamental nessa camada, fornecendo as informações necessárias ao funcionamento do sistema e executando as ações determinadas pela camada de controle. No sistema proposto, essa camada corresponde diretamente aos componentes definidos na metodologia: os manipuladores Franka Emika Panda e UR10, o robô móvel KUKA youBot, o sensor infravermelho, o sensor de visão e o dispositivo embarcado ESP32.

A camada de controle e serviços é o núcleo funcional do sistema: é nela que os dados recebidos pelos agentes são processados, as decisões são tomadas e as ações são determinadas. Afanasyev *et al.* (2019) afirmam que essa camada transforma dados em ações operacionais, e Krejčí *et al.* (2025) acrescentam que é justamente a integração de mecanismos de inteligência artificial nesse nível que permite aos robôs realizar tarefas como reconhecimento de objetos, navegação autônoma e adaptação ao ambiente com maior eficiência. No sistema proposto, essa função é exercida pelos *scripts* Python que controlam cada agente e pela inferência embarcada realizada no ESP32, cujo resultado de classificação é transmitido via MQTT e utilizado pela camada de controle como entrada para o processo decisório que determina o destino de cada caixa.

A organização em camadas não é apenas uma escolha de design, mas um reflexo de como os sistemas IoRT evoluíram. Arquiteturas modernas tendem a distribuir funções de processamento entre diferentes camadas, reduzindo a dependência de infraestruturas centralizadas e aumentando a resiliência do sistema diante de falhas parciais (VERMESAN *et al.*, 2020). Essa perspectiva é especialmente relevante para justificar a adoção do Edge AI na arquitetura proposta: ao processar as imagens localmente no ESP32, o sistema elimina a dependência de um servidor centralizado de classificação, tornando-o mais robusto e operacionalmente independente.

2.6 EDGE AI E EXECUÇÃO DE REDES NEURAIS EM DISPOSITIVOS EMBARCADOS

O processamento de inteligência artificial em dispositivos embarcados emergiu como resposta às limitações impostas por arquiteturas de inferência centralizadas. Quando um sistema depende exclusivamente de servidores remotos para executar modelos de visão computacional, cada inferência implica um ciclo de captura, transmissão, processamento e retorno de resultado, fluxo que introduz latência adicional, aumenta a dependência de conectividade e cria um ponto de falha centralizado. Nesse contexto, Musa *et al.* (2025) observam que "à medida que a computação na borda ganha espaço em diversos domínios, a demanda por modelos de aprendizado profundo leves, capazes de operar eficientemente em dispositivos com recursos restritos, cresce de forma expressiva"(MUSA *et al.*, 2025, p. 1, tradução própria).

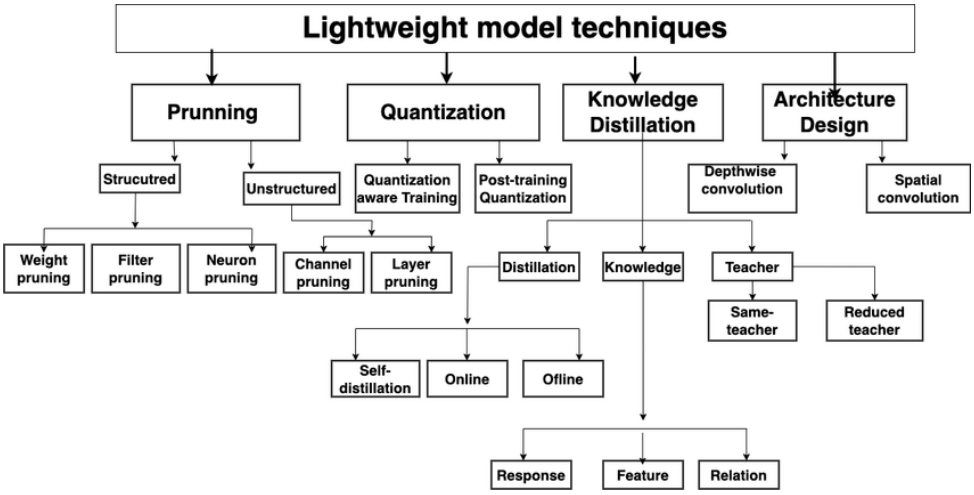
No contexto deste trabalho, essa demanda manifesta-se de forma direta: o dispositivo embarcado ESP32 deve executar a inferência localmente, classificar imagens capturadas pelo sensor de visão e transmitir apenas o resultado via MQTT. Essa abordagem elimina a necessidade de transmitir imagens brutas pela rede, reduz a latência do processo decisório e mantém o sistema funcional mesmo em situações de conectividade degradada. A relevância dessa abordagem é sintetizada por Krejčí *et al.* (2025), para quem o Edge AI em robótica "permite que os robôs executem tarefas como navegação, reconhecimento de objetos e adaptação ao ambiente em tempo real, com maior eficiência"(KREJČÍ *et al.*, 2025, p. 23, tradução própria).

A execução de redes neurais em dispositivos embarcados impõe, contudo, restrições severas. Dispositivos como o ESP32 possuem memória SRAM limitada a 512 KB e dependem de aritmética eficiente para executar inferência em tempo razoável. Redes neurais convolucionais (CNNs) convencionais, projetadas para GPUs, são inviáveis nesse ambiente sem adaptação. Zhang *et al.* (2018) destacam que a implementação de CNNs em plataformas embarcadas exige reduções no número de parâmetros e otimização das operações matemáticas, e Cruz, Lobos-Tsunekawa & Solar (2018) demonstram que arquiteturas compactas podem ser empregadas com sucesso em sistemas robóticos embarcados para tarefas de reconhecimento visual. Essas constatações justificam a escolha das arquiteturas MobileNetV2 e MobileNetV3 Small na metodologia: ambas foram projetadas especificamente para execução em dispositivos com recursos restritos, com estruturas que minimizam o número de parâmetros sem comprometer a capacidade de extração de características visuais.

Para além da escolha da arquitetura, a adaptação do modelo ao ambiente embarcado exige técnicas específicas. Musa *et al.* (2025) apresentam uma taxonomia organizada em quatro estratégias principais: *Network Pruning*, que elimina conexões e neurônios redundantes da rede treinada; *Knowledge Distillation*, na qual um modelo menor aprende a imitar o comportamento de um modelo de maior capacidade; *Architecture Design*, que consiste no desenvolvimento de arquiteturas já concebidas para execução eficiente, como as redes MobileNet; e *Quantization*, que reduz a precisão numérica dos parâmetros do modelo, substituindo representações em ponto flutuante por inteiros de menor largura de bits. O resultado prático dessas otimizações

é que os modelos resultantes alcançam "acurácia comparável à dos modelos tradicionais, com pequena redução no desempenho, mas com redução expressiva no custo computacional, no uso de memória e no consumo de energia, tornando-os adequados para aplicações em tempo real em dispositivos de borda"(MUSA *et al.*, 2025, p. 2, tradução própria). A Figura 6 representa a taxonomia dessas técnicas.

Figura 6 – Taxonomia das técnicas de otimização de modelos para Edge AI.



Fonte: Adaptado de Musa *et al.* (2025).

A quantização ocupa papel central na metodologia deste trabalho: três variantes são avaliadas para cada arquitetura, a saber, sem quantização (referência em ponto flutuante), com quantização dinâmica (pesos em int8, ativações em float32) e com quantização inteira completa (pesos, ativações, entradas e saídas em int8). Essa progressão permite comparar o impacto de cada nível de quantização sobre o tamanho do modelo, o consumo de memória e o desempenho de classificação. Musa *et al.* (2025) afirmam que a quantização inteira é a técnica que permite maior redução no uso de memória com menor custo adicional de implementação, sendo amplamente compatível com execução exclusiva em aritmética inteira, exatamente o que o ESP32 requer para operação eficiente. A ferramenta TensorFlow Lite, mencionada pelos mesmos autores como principal infraestrutura para conversão e execução de modelos embarcados, é empregada em todas as etapas de conversão descritas na metodologia.

A robustez operacional do sistema também é favorecida por essa escolha. A execução local de modelos elimina a dependência de conectividade durante o processo de inferência, o que contribui diretamente para a estabilidade do sistema em condições de rede variáveis (BADIA *et al.*, 2025). No caso deste trabalho, uma vez que o ESP32 realiza a classificação localmente e transmite apenas o resultado via MQTT, o fluxo de coordenação dos agentes robóticos continua operando mesmo que haja flutuações na qualidade da rede, com impacto restrito ao tempo de transmissão do resultado, que é uma mensagem de dimensões mínimas em comparação com a imagem original.

2.7 AVALIAÇÃO DE DESEMPENHO EM SISTEMAS IORT

A avaliação de desempenho em sistemas IoRT não pode ser reduzida à análise de componentes isolados. O comportamento global do sistema emerge da interação entre seus componentes, e é esse comportamento emergente que determina se a arquitetura proposta é viável para as condições operacionais definidas. Vermesan *et al.* (2020) afirmam que sistemas IoRT integram comunicação, processamento e coordenação como elementos interdependentes, e que a avaliação deve refletir essa interdependência: degradações na comunicação afetam a coordenação, e limitações computacionais influenciam a capacidade de resposta dos agentes.

A latência é a métrica mais diretamente associada à capacidade de coordenação em tempo real. Conforme os agentes dependem das mensagens recebidas para determinar suas próximas ações, o tempo de transmissão desde a publicação até o consumo da mensagem define o intervalo mínimo entre eventos consecutivos no fluxo operacional. Vermesan *et al.* (2020) destacam que aplicações robóticas distribuídas frequentemente operam sob restrições temporais, onde atrasos excessivos comprometem a coordenação entre agentes. Os dados de Mishra, Mishra & Kertesz (2021) quantificam esse impacto em cenários de alta carga: a diferença entre 0,87 ms (EMQX com QoS 1) e 58,48 ms (HiveMQ sob carga intensa) evidencia que a escolha de *broker* e política de QoS produz efeitos mensuráveis sobre o comportamento do sistema. Esses dados de referência fundamentam a avaliação comparativa das configurações do Eclipse Mosquitto prevista na metodologia.

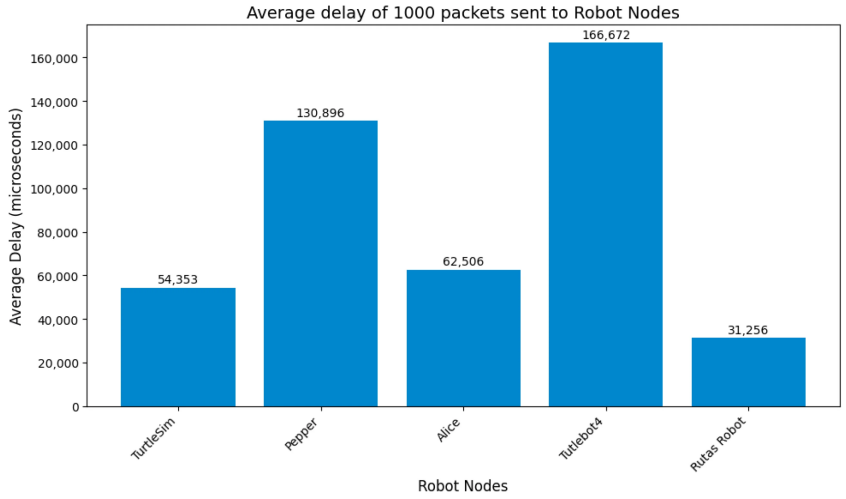
O *jitter*, definido na metodologia como o desvio padrão das latências observadas, complementa a análise ao capturar a variabilidade temporal da transmissão. Um sistema com baixa latência média, mas alto *jitter*, apresenta comportamento temporal imprevisível, o que pode comprometer a sincronização entre agentes em fluxos operacionais com dependências sequenciais estritas. Dave, Doshi & Arolkar (2020) destacam que flutuações no tempo de transmissão são um fator crítico em sistemas baseados em troca de mensagens periódicas, e Azzedin & Alhazmi (2023) reforçam que variações excessivas podem resultar em inconsistências operacionais em arquiteturas assíncronas. No sistema proposto, *jitter* elevado implicaria que o manipulador Franka receberia a notificação de disponibilidade da caixa em instantes imprevisíveis, comprometendo a fluidez do ciclo operacional.

A taxa de sucesso, proporção de ciclos operacionais concluídos em relação ao total executado, representa a dimensão de confiabilidade operacional do sistema. Em arquiteturas multiagentes nas quais os agentes dependem de eventos para iniciar suas ações, a não entrega de uma mensagem pode interromper o fluxo e impedir a conclusão do ciclo. Romeo *et al.* (2020) destacam que a cooperação entre agentes robóticos exige mecanismos de comunicação confiáveis, sendo necessário observar o comportamento do sistema em execuções repetidas para identificar falhas e inconsistências. A execução de 50 ciclos operacionais por configuração, prevista na metodologia, cumpre exatamente essa função: produzir um conjunto de observações suficiente para distinguir falhas pontuais de problemas sistêmicos.

O consumo de recursos computacionais pelo *broker* MQTT constitui a quarta dimensão de avaliação. Dispositivos embarcados e infraestruturas de baixo custo impõem limites ao uso de CPU e memória, e o *broker* não pode competir por recursos com os agentes de controle que operam na mesma máquina. Badia *et al.* (2025) demonstram que a execução de modelos de inteligência artificial em dispositivos embarcados altera o comportamento temporal e o consumo de recursos do sistema, e essa observação se estende ao *middleware*: o *broker* também consome recursos que precisam ser monitorados. O uso das ferramentas *htop* e *top* previsto na metodologia cumpre essa função, fornecendo dados que permitem avaliar a sustentabilidade da infraestrutura de comunicação nas três configurações testadas.

A Figura 7 apresenta dados comparativos de latência em um sistema IoRT com múltiplos robôs heterogêneos, ilustrando a variação desse indicador em função das plataformas envolvidas e servindo como referência para contextualizar os resultados obtidos nesta pesquisa.

Figura 7 – Comparativo de latência de publicação por plataforma robótica.



Fonte: Adaptado de Zegarra, Aranibar & Cardinale (2024).

Sorri, Mustafee & Seppänen (2022) destacam que a análise isolada de componentes não permite compreender o comportamento global de sistemas distribuídos, sendo necessário adotar abordagens que considerem simultaneamente comunicação, processamento e execução operacional. Essa perspectiva integradora fundamenta a estrutura experimental adotada neste trabalho, na qual as mesmas métricas de latência, *jitter* e taxa de sucesso são aplicadas nos dois experimentos: o de avaliação da arquitetura de comunicação sem inferência embarcada e o de avaliação do sistema integrado com o Edge AI ativo. Essa consistência metodológica é o que permite comparar os resultados e atribuir as diferenças observadas à contribuição específica do processamento embarcado.

2.8 AMBIENTES DE SIMULAÇÃO PARA SISTEMAS IORT

A validação experimental de sistemas IoRT compostos por múltiplos agentes físicos impõe desafios práticos que tornam a simulação não apenas conveniente, mas necessária. Sistemas robóticos interagem com o ambiente físico, operam sob restrições de tempo real e dependem de infraestrutura de comunicação funcional, condições que, em conjunto, tornam a validação em *hardware* físico onerosa, demorada e sujeita a variáveis de difícil controle. Takaya *et al.* (2016) afirmam que ambientes simulados permitem representar robôs, sensores e atuadores em cenários controlados, possibilitando o teste de arquiteturas sem os riscos associados à implementação direta em sistemas físicos e com capacidade de repetir as condições experimentais de forma consistente.

No contexto específico de sistemas IoRT, a simulação oferece uma vantagem adicional: permite avaliar o comportamento da infraestrutura de comunicação em condições controladas antes de expô-la às variabilidades do ambiente real. Nieto *et al.* (2025) destacam que ambientes simulados permitem controlar variáveis experimentais, repetir execuções com as mesmas condições iniciais e observar o comportamento do sistema sob diferentes parâmetros operacionais, capacidade de reprodutibilidade que é um requisito central para validação científica. Em um sistema no qual a latência de comunicação é uma métrica de interesse, a capacidade de isolar essa variável das outras fontes de variabilidade é especialmente valiosa.

O CoppeliaSim foi selecionado como ambiente de simulação neste trabalho em função de características específicas que o diferenciam de outros simuladores. Bogaerts *et al.* (2020) descrevem o ambiente como:

> O CoppeliaSim é um ambiente de simulação de robôs utilizado para prototipagem, desenvolvimento e verificação de sistemas e algoritmos robóticos (BOGAERTS *et al.*, 2020, tradução própria).

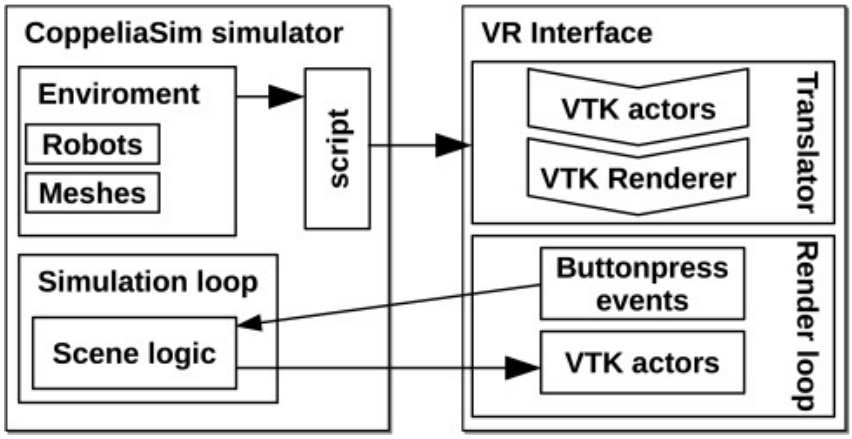
Além das capacidades gerais de simulação robótica, o CoppeliaSim apresenta uma característica arquitetural relevante para este trabalho: ele permite implementar a lógica de controle externamente ao simulador, por meio de *scripts* e aplicações conectadas por uma interface de comunicação. Isso significa que os *scripts* Python que controlam os manipuladores e o robô móvel operam fora do ambiente de simulação, comunicando-se com os modelos virtuais exatamente como fariam com os robôs físicos. Essa característica é fundamental para que o *middleware* MQTT possa ser inserido de forma genuína na arquitetura, com os *scripts* de controle utilizando o *broker* para coordenar os agentes simulados da mesma forma que coordenariam agentes físicos. Astudillo & Missael (2022) demonstrou essa capacidade ao implementar um algoritmo de consenso para sistemas multiagentes com lógica de controle desenvolvida externamente ao CoppeliaSim.

O simulador também suporta a simulação simultânea de múltiplos robôs em um mesmo ambiente dinâmico. Bogaerts *et al.* (2020) afirmam que essa capacidade permite avaliar a coordenação entre agentes e observar o comportamento emergente do sistema sob condições

operacionais definidas. Marian *et al.* (2020) complementam que a simulação permite avaliar aspectos relacionados à comunicação, coordenação e sincronização entre agentes distribuídos, não apenas a dinâmica local de cada robô individualmente. Essa perspectiva está alinhada ao objetivo experimental desta pesquisa: não se trata de validar o controle de um único robô, mas o comportamento do sistema como um todo, considerando a coordenação entre os três agentes robóticos, o fluxo de mensagens MQTT e a integração com o dispositivo embarcado físico ESP32.

Gonzalez, Coronado & Yamanobe (2025) mapearam diferentes simuladores utilizados em sistemas IoRT, identificando como critérios de seleção o suporte a protocolos de comunicação, a integração com *middlewares* e a capacidade de representar múltiplos agentes simultaneamente. O CoppeliaSim atende a esses três critérios no contexto deste trabalho: opera com integração nativa às bibliotecas Python utilizadas na arquitetura, suporta os modelos dos robôs Franka Emika Panda, UR10 e KUKA youBot, e permite a execução simultânea dos três agentes em um único ambiente com elementos funcionais como esteira transportadora, sensor infravermelho e estante de armazenamento. A Figura 8 ilustra a estrutura do ambiente de simulação e sua integração com os sistemas externos de controle.

Figura 8 – Arquitetura do CoppeliaSim integrado a sistemas de controle externos.



Fonte: Adaptado de Bogaerts *et al.* (2020).

Takaya *et al.* (2016) reforçam ainda que a simulação permite testar arquiteturas sem utilizar robôs físicos, reduzindo custos e riscos durante o desenvolvimento, característica particularmente relevante em pesquisas acadêmicas com recursos físicos limitados. No contexto deste trabalho, o único elemento físico do sistema é o ESP32: todos os agentes robóticos e os sensores são virtuais, o que permite concentrar os esforços de validação na arquitetura de comunicação e no comportamento da inferência embarcada, sem que variáveis do *hardware* físico dos robôs contaminem as medições. Essa separação deliberada entre o que é simulado e o que é físico é uma decisão metodológica fundamentada diretamente nas recomendações da literatura sobre sistemas IoRT com componentes heterogêneos.

3 METODOLOGIA

A presente pesquisa caracteriza-se como experimental e quantitativa. A natureza experimental é evidenciada pela manipulação controlada de variáveis (avaliação de três configurações distintas de *broker* MQTT e comparação de diferentes arquiteturas de redes neurais), procedimento fundamental para validar soluções tecnológicas em ambientes controlados (GIL, 2022). O caráter quantitativo reside na coleta e análise sistemática de dados numéricos, incluindo taxa de sucesso, latência, *jitter*, acurácia e consumo de memória, que permitem uma avaliação objetiva da arquitetura proposta (GIL, 2022).

O método foi organizado em seis etapas encadeadas: (1) construção do cenário experimental no simulador CoppeliaSim, que estabelece o ambiente controlado e os agentes robóticos sobre os quais todo o trabalho se apoia; (2) definição da arquitetura de *software* em quatro camadas, estruturando os componentes e suas responsabilidades; (3) configuração do *middleware* MQTT, que viabiliza a troca de mensagens entre os agentes; (4) avaliação experimental da comunicação, medindo latência, *jitter* e taxa de sucesso nas três configurações de *broker*; (5) desenvolvimento, treinamento e conversão dos modelos de redes neurais para o formato *TensorFlow Lite*; e (6) integração da inferência embarcada no ESP32 ao fluxo operacional, com avaliação do sistema completo. Cada etapa depende diretamente da anterior: sem o cenário definido não há fluxo a coordenar; sem o *middleware* configurado não há como medir comunicação; e sem os modelos convertidos e validados no *hardware* não há inferência embarcada a integrar.

3.1 CONSTRUÇÃO DO CENÁRIO EXPERIMENTAL

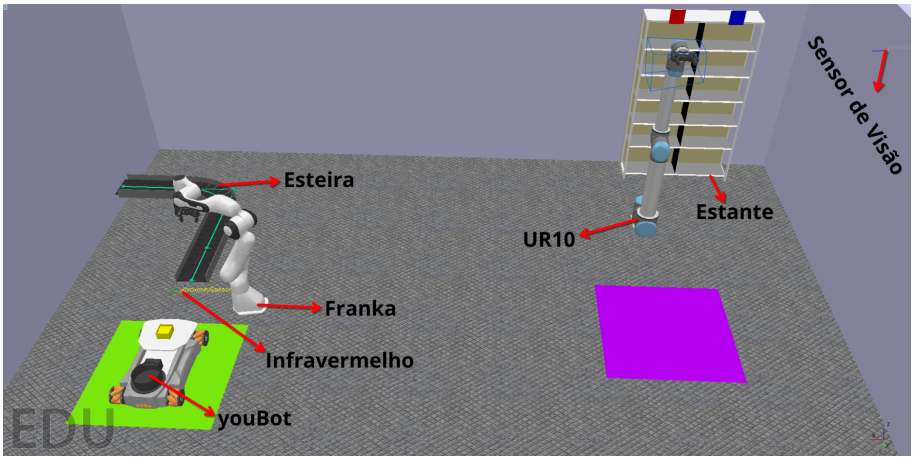
O cenário industrial simulado foi desenvolvido para execução das tarefas de coleta, transporte e armazenamento de caixas, estabelecendo um ambiente experimental controlado para avaliação da arquitetura de comunicação e da colaboração entre os agentes robóticos. O cenário representa um fluxo logístico estruturado, composto por eventos sequenciais e dependentes, o que permite analisar a troca de mensagens entre os componentes e a organização das tarefas no sistema.

A implementação foi realizada no simulador CoppeliaSim EDU (versão 4.10.0), executado em sistema Linux, utilizando modelos disponíveis do manipulador Franka Emika Panda, do robô móvel KUKA youBot e do manipulador UR10. O ambiente foi configurado com uma esteira transportadora atuada, um sensor infravermelho posicionado em sua extremidade, um sensor de visão para captura de imagens e uma estante destinada ao armazenamento das caixas. A lógica de controle foi implementada em Python (versão 3.10 no ambiente de treinamento e versão 3.13.5 no ambiente local), viabilizando a comunicação entre os dispositivos e definindo o fluxo de execução das tarefas.

O sensor infravermelho identifica a presença de uma caixa ao final da esteira e sinaliza

sua disponibilidade para coleta. A partir desse evento, o manipulador Franka realiza a coleta e posiciona a caixa sobre o robô móvel youBot, que realiza o transporte até o ponto de entrega. O manipulador UR10 efetua a coleta final e posiciona a caixa na estante, concluindo o fluxo operacional. O sensor de visão foi posicionado próximo à área de atuação do UR10, possibilitando a captura das imagens utilizadas nas etapas de classificação embarcada. O ciclo completo de coleta, transporte e armazenamento foi validado antes dos experimentos. A Figura 9 apresenta a configuração final do cenário, com a identificação dos agentes e sua disposição no ambiente simulado.

Figura 9 – Cenário de simulação CoppeliaSim, com identificação dos agentes



Fonte: Próprio autor (2026).

O sistema é composto pelos seguintes agentes e dispositivos: (a) manipulador robótico Franka Emika Panda, responsável pela coleta das caixas na esteira; (b) robô móvel KUKA youBot, responsável pelo transporte das caixas entre as zonas de coleta e armazenamento; (c) manipulador robótico UR10, responsável pela coleta final e pelo posicionamento das caixas na estante; (d) sensor infravermelho, posicionado na extremidade da esteira para detectar a presença de caixas; (e) sensor de visão, posicionado próximo à área de atuação do UR10 para captura das imagens enviadas ao ESP32; e (f) dispositivo embarcado ESP32-S3-WROOM-1-N16R8, responsável pela execução local dos modelos de classificação e pela transmissão do resultado via MQTT. O diagrama de atividades completo do sistema, ilustrando o fluxo operacional e a distribuição de responsabilidades entre esses agentes, está disponível no Apêndice F.

3.2 DEFINIÇÃO DA ARQUITETURA DE SOFTWARE

A arquitetura de *software* do sistema colaborativo IoRT foi estruturada em quatro camadas, cada uma com um conjunto específico de responsabilidades, separando os elementos responsáveis pela execução das ações, pela comunicação e pelo processamento das informações.

A Camada Física é composta pelos manipuladores Franka Emika Panda e UR10, pelo robô móvel KUKA youBot, pelos sensores e pelo dispositivo embarcado ESP32-S3-WROOM-1-N16R8, representando os elementos responsáveis pela execução das ações e pela interação com o ambiente. A Camada de Rede representa os mecanismos responsáveis pela conectividade entre os dispositivos via Wi-Fi, viabilizando o transporte das mensagens. A Camada de Comunicação define o *middleware* responsável pela troca de mensagens entre os componentes, utilizando o protocolo MQTT no modelo publicação e assinatura. A Camada de Controle reúne os *scripts* responsáveis pelo processamento das informações e pela execução das ações definidas no sistema, utilizando os dados recebidos por meio da comunicação.

A implementação foi realizada em Python, com integração ao CoppeliaSim EDU e uso de clientes MQTT para comunicação entre módulos. Foram desenvolvidos *scripts* independentes para controle dos manipuladores Franka Emika Panda e UR10, navegação do robô móvel KUKA youBot e gerenciamento dos sensores. A comunicação foi estruturada por tópicos MQTT, de modo que os componentes publicam estados e consomem eventos necessários à execução do fluxo operacional. A Figura 10 apresenta o diagrama completo da arquitetura com a identificação de cada camada e seus componentes.

Figura 10 – Arquitetura IoRT multiagente de quatro camadas.



Fonte: Próprio autor (2026).

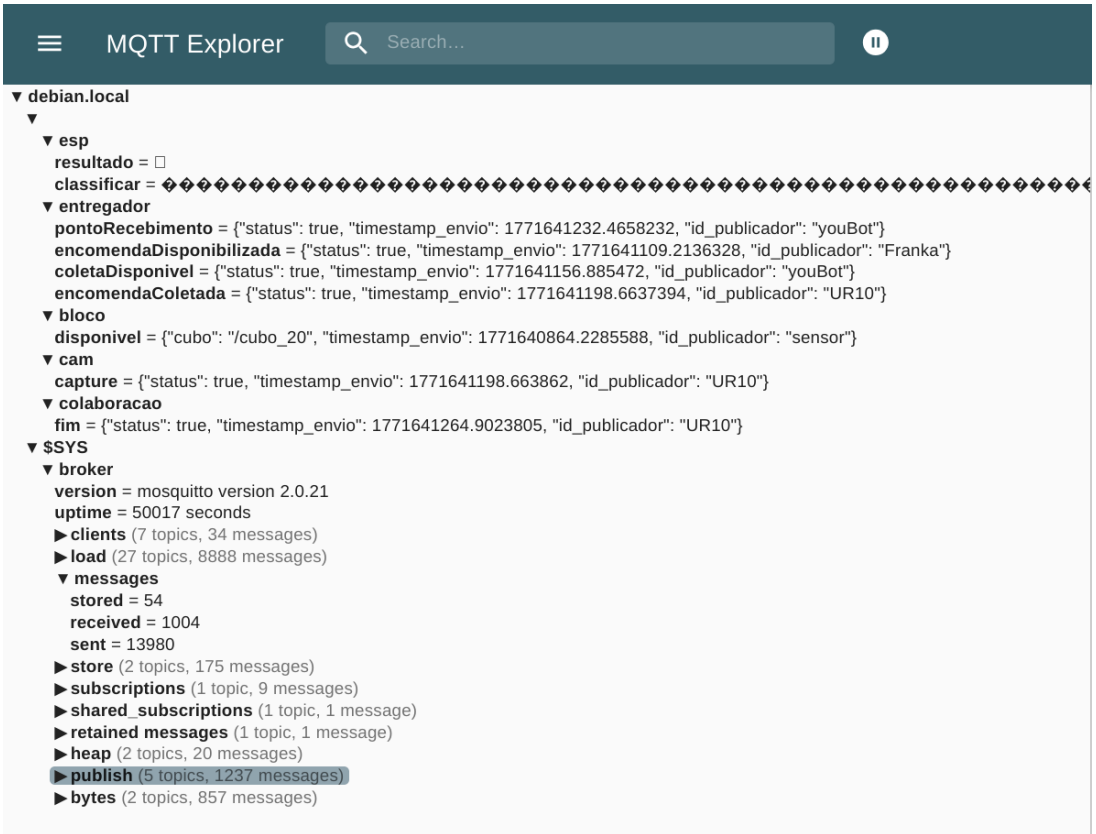
3.3 CONFIGURAÇÃO DO MIDDLEWARE MQTT

A Camada de Comunicação foi implementada para viabilizar a troca de mensagens entre robôs, sensores, dispositivo embarcado e módulos de controle. O objetivo é transmitir eventos do fluxo operacional entre os agentes, permitindo que cada componente execute suas ações com base nas informações recebidas. O uso de um *middleware* de mensagens mantém os módulos desacoplados, com interação baseada em eventos.

A comunicação foi implementada com o protocolo MQTT, por meio do *broker* Eclipse Mosquitto e da biblioteca *paho-mqtt* em Python. O modelo de publicação e assinatura foi adotado: cada componente publica mensagens em tópicos específicos e consome eventos por meio de assinaturas. Todas as mensagens foram transmitidas com QoS 1 (entrega garantida de pelo menos uma vez), equilibrando confiabilidade e *overhead* de transmissão. O *broker* foi configurado com autenticação por usuário e senha, com acesso anônimo desativado.

O funcionamento do *middleware* foi acompanhado com ferramentas de monitoramento e inspeção, incluindo MQTT Explorer (visualização de tópicos e mensagens) e Wireshark (tráfego de rede). O *broker* foi executado em três configurações: local, em nuvem (instância Amazon EC2 com Debian 12) e em dispositivo de borda Intel Edison, possibilitando comparar o comportamento da comunicação sob diferentes infraestruturas. A Figura 11 apresenta a estrutura de tópicos do sistema capturada pelo MQTT Explorer durante a execução do fluxo operacional.

Figura 11 – Estrutura de tópicos MQTT do sistema capturada pelo MQTT Explorer.


MQTT Explorer
Search...
▼ debian.local
▼
▼ esp
resultado = □
classificar = ▯▯▯

3.4 AVALIAÇÃO EXPERIMENTAL DA COMUNICAÇÃO

A avaliação quantitativa da arquitetura de comunicação mediu o comportamento temporal da troca de mensagens durante a execução do fluxo colaborativo, sem processamento de inteligência artificial embarcada. O objetivo foi registrar métricas relacionadas ao tempo de transmissão das mensagens e ao desempenho durante os ciclos operacionais.

Os experimentos foram conduzidos por meio da execução repetida do ciclo completo de coleta, transporte e armazenamento no ambiente simulado. Cada configuração do *broker* foi executada cinco vezes, com 50 ciclos operacionais por execução. Em cada ciclo, foram registrados os instantes de envio e recebimento das mensagens, gerando os dados necessários para o cálculo das métricas.

A latência (L) foi utilizada como métrica para medir o tempo de transmissão das mensagens entre os componentes do sistema, sendo definida como a diferença entre o instante de recebimento (t_f) e o instante de envio (t_0), conforme apresentado na Equação 3.1.

$$L = t_f - t_0 \tag{3.1}$$

onde t_0 representa o instante de envio e t_f o instante de recebimento da mensagem. Essa métrica quantifica o intervalo de tempo associado à transmissão das mensagens no sistema.

O *jitter* (J) foi utilizado para medir a variação dos valores de latência obtidos durante as transmissões, sendo definido como o desvio padrão das latências observadas, conforme apresentado na Equação 3.2.

$$J = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(L_i - \bar{L})^2} \tag{3.2}$$

onde L_i representa a latência individual de cada mensagem, $\bar{L}$ representa a latência média e N representa o número total de medições. Essa métrica quantifica a variação temporal da transmissão das mensagens ao longo da execução dos ciclos operacionais.

A taxa de sucesso (S) foi utilizada para registrar a proporção de ciclos operacionais concluídos em relação ao número total de ciclos executados, conforme apresentado na Equação 3.3.

$$S = \frac{N_c}{N_t} \times 100\% \tag{3.3}$$

onde N_c representa o número de ciclos concluídos e N_t representa o número total de ciclos executados.

O uso de recursos computacionais do *broker* Eclipse Mosquitto também foi monitorado, incluindo consumo de CPU e memória, utilizando as ferramentas *htop* e *top*.

3.5 TREINAMENTO DOS MODELOS

O desenvolvimento e treinamento dos modelos de redes neurais convolucionais foi realizado para classificação de imagens em três categorias: azul, vermelho e *background*. Para viabilizar a execução em dispositivos embarcados com recursos limitados, as imagens de entrada foram redimensionadas para 32×32 *pixels* no formato RGB com três canais de cor, reduzindo a quantidade de dados processados por inferência.

Foram utilizadas três arquiteturas distintas: MobileNetV2, MobileNetV3 Small e uma rede neural convolucional autoral. As arquiteturas MobileNetV2 e MobileNetV3 Small foram escolhidas por apresentarem estruturas otimizadas para redução do número de parâmetros e operações, mantendo capacidade de extração de características em imagens. A rede convolucional autoral foi desenvolvida sem seguir arquitetura predefinida, sendo composta por uma estrutura sequencial formada por duas camadas convolucionais com funções de ativação ReLU, cada uma seguida por uma camada de redução dimensional, uma etapa de transformação para vetor unidimensional, uma camada totalmente conectada com função de ativação ReLU, uma camada de regularização *Dropout* e uma camada de saída com três neurônios e função de ativação *Softmax*, correspondentes às classes definidas. Os hiperparâmetros de treinamento de cada arquitetura estão descritos na Tabela 1.

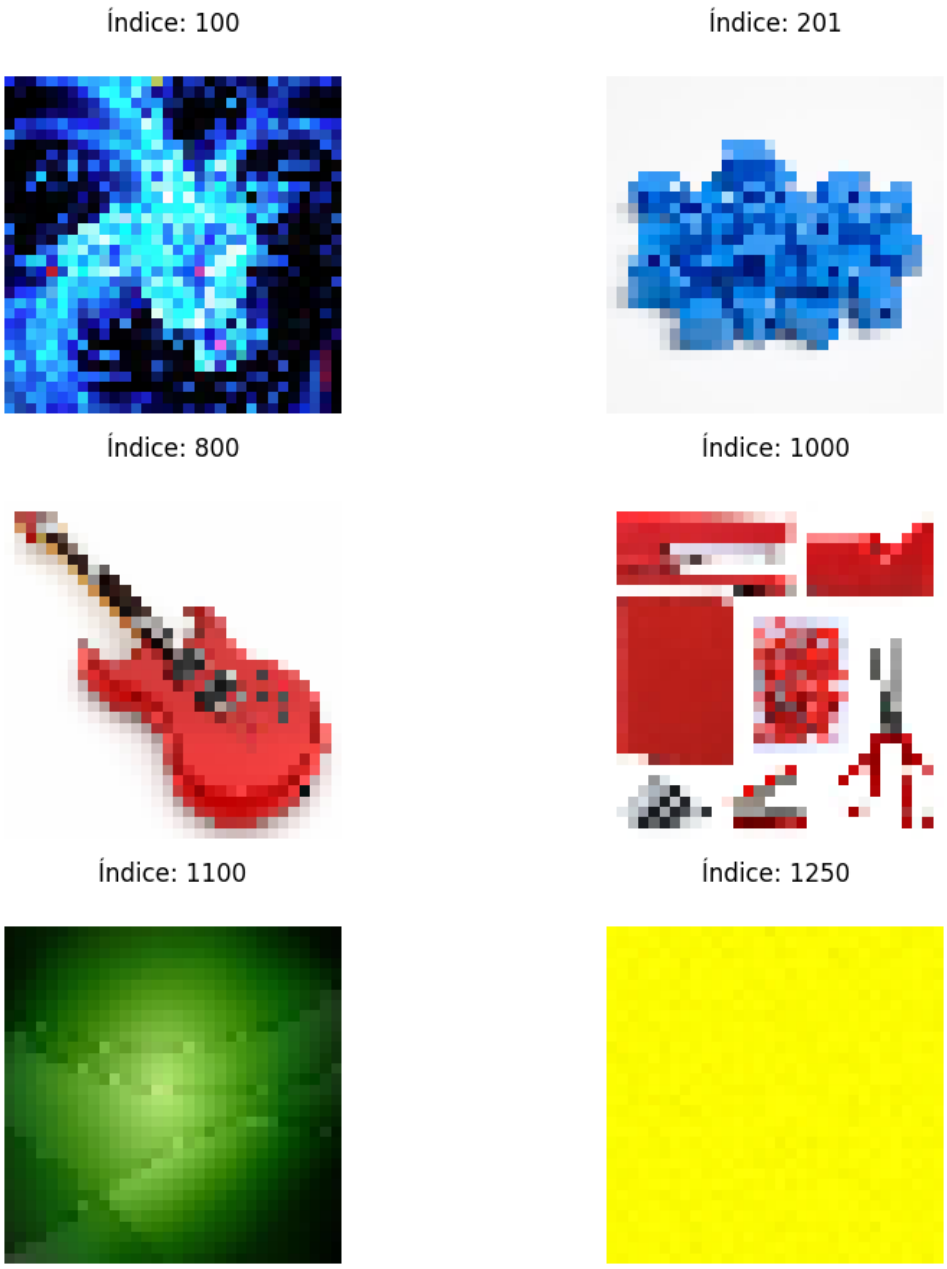
A base de dados foi composta por 1.656 imagens coletadas especificamente para o treinamento dos modelos, sendo 540 imagens contendo objetos predominantemente vermelhos, 540 contendo objetos predominantemente azuis e 576 imagens contendo diferentes objetos e contextos visuais sem predominância dessas cores, utilizadas como classe *background*. O conjunto de dados foi dividido em três subconjuntos: 70% destinado ao treinamento, 15% à validação e 15% ao teste, com amostragem estratificada para preservar a proporção entre as classes. A Figura 12 apresenta exemplos representativos de cada classe.

Tabela 1 – Hiperparâmetros de treinamento por arquitetura

Parâmetro	**MobileNetV2**	**MobileNetV3 Small**	**CNN Autoral**
Otimizador	Adam	Adam	Adam
Learning rate (fase 1)	0,001	0,001	1e-6
Learning rate (fase 2)	2e-5	2e-5	—
Épocas (fase 1)	50	50	—
Épocas (fase 2 / total)	270	600	350
Batch size	16	16	8
Dropout	0,4	0,4	0,3
Ativação saída	*Softmax*	*Softmax*	*Softmax*
Regularização	L2 (0,001)	L2 (0,001)	—

Fonte: elaborada pelo autor (2026).

Figura 12 – Exemplos de imagens do conjunto de dados nas três classes de classificação.



Fonte: elaborada pelo autor (2026).

O treinamento foi conduzido utilizando as bibliotecas TensorFlow e Keras (versão 2.17.0), juntamente com a linguagem Python (versão 3.10), no ambiente Google Colaboratory, que disponibiliza infraestrutura baseada em Linux com aceleração por *hardware* (GPU NVIDIA T4), seguindo o paradigma de aprendizado supervisionado. As arquiteturas MobileNetV2 e MobileNetV3 Small foram ajustadas por meio do procedimento de *fine-tuning*, adaptando seus parâmetros à tarefa de classificação definida. A rede convolucional autoral foi treinada a partir de parâmetros inicializados aleatoriamente. A Figura 13 apresenta as definições das três arquiteturas utilizadas e a Figura 14 ilustra o procedimento de ajuste fino aplicado às arquiteturas baseadas em transferência de aprendizado.

Figura 13 – Definição das arquiteturas dos modelos de classificação.

```
modelV3 = MobileNetV3Small(
  weights='imagenet',
  include_top=False,
  input_shape=(32, 32, 3))
modelV3.trainable = False

modeloV3 = Sequential([
  modelV3,
  GlobalAveragePooling2D(),
  Dense(32,
    activation=LeakyReLU
    (0.01),
    kernel_regularizer=l2
    (0.001)),
  Dropout(0.4),
  Dense(3,
    activation='softmax',
    kernel_regularizer=l2
    (0.001))])

modeloV3.compile(
  optimizer=Adam(0.001),
  loss='sparse_categorical_'
       'crossentropy',
  metrics=['accuracy'])
```

(a) MobileNetV3 Small

```
modelV2 = MobileNetV2(
  weights='imagenet',
  include_top=False,
  input_shape=(32, 32, 3))
modelV2.trainable = False

modeloV2 = Sequential([
  modelV2,
  GlobalAveragePooling2D(),
  Dense(32,
    activation=LeakyReLU
    (0.01),
    kernel_regularizer=l2
    (0.001)),
  Dropout(0.4),
  Dense(3,
    activation='softmax',
    kernel_regularizer=l2
    (0.001))])

modeloV2.compile(
  optimizer=Adam(0.001),
  loss='sparse_categorical_'
       'crossentropy',
  metrics=['accuracy'])
```

(b) MobileNetV2

```
model = Sequential([
  Input(shape=(32, 32, 3)),
  Conv2D(32, (3,3),
    activation='relu'),
  MaxPooling2D((2, 2)),
  Conv2D(64, (3,3),
    activation='relu'),
  MaxPooling2D((2, 2)),
  Flatten(),
  Dense(64, activation='relu'),
  Dropout(0.3),
  Dense(3,
    activation='softmax')])

model.compile(
  optimizer=Adam(1e-6),
  loss='sparse_categorical_'
       'crossentropy',
  metrics=['accuracy'])

model.fit(X_treino, y_treino,
  batch_size=8, epochs=350,
  validation_data=(
    X_teste, y_teste),
  callbacks=[checkpoint])
```

(c) CNN Autoral

Fonte: elaborada pelo autor (2026).

Figura 14 – Procedimento de ajuste fino em duas fases (MobileNetV2 e MobileNetV3 Small).

```
# Fase 1 -- Treinamento da cabeca com a base congelada (50 epocas, lr=0.001)
H1 = modeloV3.fit(X_treino, y_treino, batch_size=16, epochs=50,
                  validation_data=(X_val, y_val), callbacks=[checkpoint1])

modeloV3.load_weights('modeloV3.keras')  # restaura melhores pesos da fase 1

# Fase 2 -- Descongelamento parcial e ajuste fino (lr=2e-5)
modelV3.trainable = False
for layer in modeloV3.layers[-30:]:     # descongelando as 30 ultimas camadas
    layer.trainable = True

modeloV3.compile(optimizer=Adam(2e-5),
                 loss='sparse_categorical_crossentropy',
                 metrics=['accuracy'])

# V3: 600 epocas  |  V2: 270 epocas
H2 = modeloV3.fit(X_treino, y_treino, batch_size=16, epochs=600,
                  validation_data=(X_val, y_val), callbacks=[checkpoint2])
```

Fonte: elaborada pelo autor (2026).

3.6 CONVERSÃO E ADAPTAÇÃO PARA EXECUÇÃO EMBARCADA

Os modelos treinados foram convertidos para um formato compatível com execução em microcontroladores, viabilizando sua utilização no dispositivo embarcado ESP32. Essa conversão é necessária porque os modelos treinados em ambiente convencional utilizam representações em ponto flutuante e estruturas que não são diretamente compatíveis com os recursos de memória e processamento disponíveis em sistemas embarcados.

A conversão foi realizada com o TensorFlow Lite Converter (versão 2.17.0), que transforma modelos treinados no formato Keras em arquivos no formato *TensorFlow Lite* (.tflite), compatível com a biblioteca *TensorFlow Lite Micro* (versão 1.3.5). Três variantes de conversão foram geradas para cada arquitetura: sem quantização (ponto flutuante de referência), com quantização dinâmica (pesos em int8, ativações em float32) e com quantização inteira completa (pesos, ativações, entradas e saídas em int8). Para a quantização inteira, um conjunto representativo de amostras do conjunto de teste foi utilizado durante o processo de calibração do conversor.

Figura 15 – Código de conversão dos modelos treinados para o formato *TensorFlow Lite*.

```
conversor = (
  tf.lite.TFLiteConverter
  .from_keras_model(
    modelo_test))

modelo_lite = (
  conversor.convert())

with open(
  'modelo_Normal.tflite',
  'wb') as f:
  f.write(modelo_lite)
```

(a) Sem quantização

```
conversor = (
  tf.lite.TFLiteConverter
  .from_keras_model(
    modelo_test))
conversor.optimizations =
    [
  tf.lite.Optimize.DEFAULT
    ]

modelo_lite = (
  conversor.convert())

with open(
  'modelo_Dinamica.tflite'
    ,
  'wb') as f:
  f.write(modelo_lite)
```

(b) Quantização dinâmica

```
def representative_dataset():
  for i in range(
      min(250, len(X_teste))):
    img = X_teste[i].astype(
      np.float32)
    img = np.expand_dims(
      img, axis=0)
    if isinstance(img, tf.Tensor):
      img = img.numpy()
    yield [img]

converter = (
  tf.lite.TFLiteConverter
  .from_keras_model(modelo_test))
converter.optimizations = [
  tf.lite.Optimize.DEFAULT]
converter.representative_dataset = (
  representative_dataset)
converter.target_spec.supported_ops
    =[
  tf.lite.OpsSet.
    TFLITE_BUILTINS_INT8]
converter.inference_input_type = (
  tf.int8)
converter.inference_output_type = (
  tf.int8)

modelo_quantizado = (
  converter.convert())
with open(
  "modelo_Inteira.tflite",
  "wb") as f:
  f.write(modelo_quantizado)
```

(c) Quantização inteira

Fonte: adaptado de Google LLC (2024).

Após a conversão, os modelos foram armazenados no formato .tflite e convertidos para representação em *arrays* em linguagem C/C++ por meio da ferramenta *xxd*, permitindo sua incorporação direta ao *firmware* e sua execução pelo interpretador embarcado.

3.7 EMBARQUE NO ESP32 E VERIFICAÇÃO DE VIABILIDADE

Os modelos convertidos foram implantados no dispositivo embarcado ESP32-S3-WROOM-1-N16R8 (Dual-Core Xtensa LX7, 240 MHz), cujas especificações de memória impõem limitações diretas ao embarque: 512 KB de SRAM interna para execução do interpreta-

dor e alocação dos tensores, 16 MB de memória Flash para armazenamento do *firmware* e dos modelos, e 8 MB de PSRAM como memória externa auxiliar. A compatibilidade de cada modelo com essas restrições precisou ser verificada antes da integração, uma vez que a conversão para o formato *TensorFlow Lite* não garante por si só a execução no microcontrolador.

Inicialmente, o custo de memória de cada modelo foi medido em ambiente Linux, utilizando a biblioteca *TensorFlow Lite Micro* com os modelos carregados como *arrays* em C/C++. O interpretador foi inicializado com uma arena de 1 MB e a função `arena_used_bytes()` registrou o consumo real de cada modelo. O Figura 16 apresenta o procedimento utilizado para essa medição.

Figura 16 – Medição do consumo de arena de tensores com *TensorFlow Lite Micro*.

```
// Arena maxima para medicao (1 MB em ambiente Linux)
constexpr int kTensorArenaSize = 1024 * 1024;
static uint8_t tensor_arena[kTensorArenaSize];

size_t medir_modelo(const ModeloInfo& info) {
    // Carrega o modelo a partir do array C/C++
    const tflite::Model* model = tflite::GetModel(info.data);

    // Zera a arena antes de cada medicao
    memset(tensor_arena, 0, kTensorArenaSize);

    // Configura o resolver com as operacoes necessarias
    tflite::MicroMutableOpResolver<25> resolver;
    info.build_resolver(resolver);

    // Inicializa o interpretador com a arena
    tflite::MicroInterpreter interpreter(
        model, resolver, tensor_arena,
        kTensorArenaSize, error_reporter);

    // Aloca os tensores e mede o consumo real
    TfLiteStatus status = interpreter.AllocateTensors();
    if (status != kTfLiteOk) return 0;

    size_t arena_used = interpreter.arena_used_bytes();

    printf("Arena usada (bytes): %u\n", (unsigned)arena_used);
    printf("Input tensor bytes:  %u\n",
           (unsigned)interpreter.input(0)->bytes);
    printf("Output tensor bytes: %u\n",
           (unsigned)interpreter.output(0)->bytes);

    return arena_used;
}
```

Fonte: elaborada pelo autor (2026).

Um modelo foi considerado viável quando atendeu simultaneamente aos requisitos de memória e armazenamento do dispositivo: capacidade de alocação da arena necessária na SRAM interna (ou PSRAM, como alternativa) e tamanho total do binário compilado dentro dos limites da partição Flash disponível. Para viabilizar essa configuração, foi utilizado o sistema de configuração do ESP-IDF por meio da ferramenta *menuconfig*, ajustando os parâmetros associados ao tamanho máximo do *firmware* e às regiões de memória utilizadas pelo sistema.

Após a verificação dos requisitos, os modelos viáveis foram incorporados ao *firmware*

compilado com o *framework* ESP-IDF (versão 5.5.2), adicionando a dependência `espressif/esp-tflite-micro` (versão 1.3.5) por meio do gerenciador de componentes nativo. As especificações de *hardware* do dispositivo utilizado estão apresentadas na Tabela 2.

Tabela 2 – Especificações do hardware embarcado utilizado na execução dos modelos

Componente	**Descrição**	**Capacidade**
Microcontrolador	ESP32-S3-WROOM-1-N16R8	Dual-Core até 240 MHz
CPU	Arquitetura Xtensa LX7 de 32 bits com suporte a instruções vetoriais	Processamento com aceleração
SRAM Interna	Memória estática utilizada para execução e alocação da arena de tensores	512 KB
Memória Flash	Armazenamento persistente para firmware e modelos	16 MB
PSRAM	Memória externa auxiliar para expansão da capacidade de execução	8 MB
Conectividade	Interface de comunicação sem fio integrada	Wi-Fi 802.11 b/g/n (2,4 GHz)

Fonte: elaborada pelo autor (2026).

3.8 INTEGRAÇÃO COM O AMBIENTE SIMULADO E CICLO DE DECISÃO E ATUAÇÃO

Os modelos viáveis foram integrados ao sistema colaborativo, incorporando o processamento de classificação ao fluxo operacional e estabelecendo a comunicação entre o sensor de visão, o dispositivo embarcado e o módulo de controle responsável pelas ações do manipulador.

No ambiente de simulação, o manipulador UR10 coleta a caixa transportada pelo robô móvel youBot e a posiciona próxima ao sensor de visão. Nessa posição, o sensor captura uma imagem e a envia ao dispositivo embarcado ESP32-S3-WROOM-1-N16R8 via MQTT. O ESP32 executa a inferência local com o interpretador *TensorFlow Lite Micro*, classifica a caixa e publica o resultado no *broker*. O módulo de controle consome esse resultado e determina o destino final da caixa na estante.

O procedimento experimental foi conduzido em cinco execuções independentes, cada uma composta por 40 operações de classificação, correspondentes à manipulação de caixas das classes azul e vermelho. Durante as execuções, foram coletados os instantes associados às etapas de captura da imagem, transmissão ao dispositivo embarcado e recebimento do resultado da inferência. As métricas de latência, *jitter* e taxa de sucesso foram obtidas com o mesmo procedimento de medição definido na avaliação da comunicação sem inferência embarcada, mantendo a consistência entre os experimentos.

4 RESULTADOS

Este capítulo apresenta os dados coletados nos experimentos realizados para avaliar a arquitetura IoRT de quatro camadas proposta. Os experimentos foram organizados em quatro blocos: (1) desempenho do *middleware* MQTT nas três configurações de *broker*, com registro de latência, *jitter* e taxa de sucesso; (2) desempenho dos modelos de classificação no conjunto de teste e impacto da conversão e quantização para o formato *TensorFlow Lite*; (3) viabilidade de embarque das arquiteturas no ESP32-S3, considerando os requisitos de memória Flash e arena de execução; e (4) desempenho do sistema integrado com a inferência embarcada ativa, avaliando o comportamento do fluxo colaborativo com a classificação executada no microcontrolador.

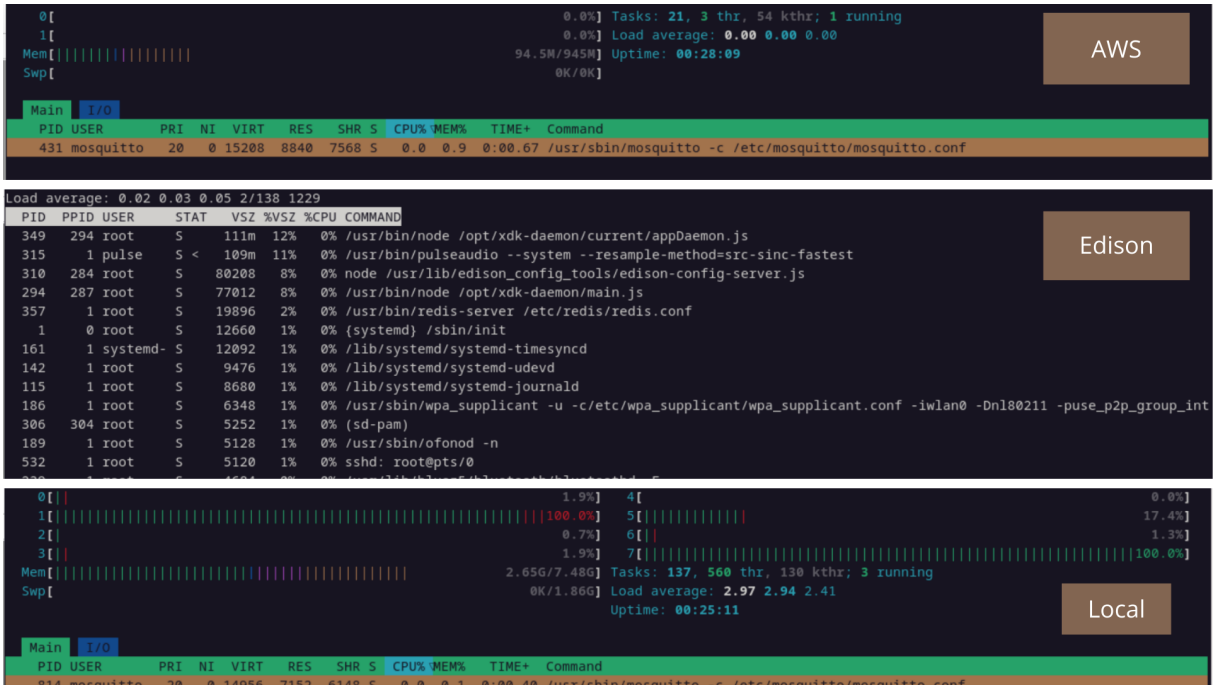
4.1 DESEMPENHO DO MIDDLEWARE MQTT

O desempenho do protocolo MQTT foi avaliado com o *broker* Eclipse Mosquitto, com monitoramento do tráfego de rede e do consumo de recursos computacionais durante a operação do sistema. O tráfego foi analisado com o Wireshark, permitindo registrar as publicações, assinaturas e mensagens transmitidas entre os agentes ao longo da execução.

Nos três ambientes, o volume de pacotes por minuto apresentou um pico inicial elevado, associado à fase de conexão e assinatura dos clientes MQTT ao iniciar a simulação. No ambiente local, esse pico atingiu aproximadamente 30 pacotes por minuto, reduzindo-se rapidamente para uma faixa de operação entre 10 e 16 pacotes por minuto ao longo de todo o período de execução (cerca de 1.600 segundos). No Intel Edison, o pico inicial ficou em torno de 22 pacotes por minuto, estabilizando entre 10 e 16 pacotes, com padrão oscilatório regular associado às atividades dos agentes. No ambiente AWS, o comportamento foi semelhante em amplitude, com os valores oscilando entre 10 e 17 pacotes por minuto e oscilações mais frequentes ao longo do tempo de execução. Em nenhum dos três ambientes foram identificadas interrupções, quedas abruptas ou séries de retransmissões que indicassem perda de mensagens.

O consumo de recursos do *broker* foi monitorado com as ferramentas *htop* e *top* durante as execuções do ciclo colaborativo, registrando percentual de CPU, consumo de memória residente (RES) e carga média do sistema (*load average*) em cada um dos três ambientes. O monitoramento foi realizado com o Mosquitto em operação ativa, recebendo e distribuindo mensagens dos agentes simulados, de forma a capturar o impacto real do *broker* sobre os recursos da máquina hospedeira. A Figura 17 apresenta os valores registrados nos três ambientes.

Figura 17 – Consumo de recursos do broker Mosquitto nos ambientes AWS, Edison e local.



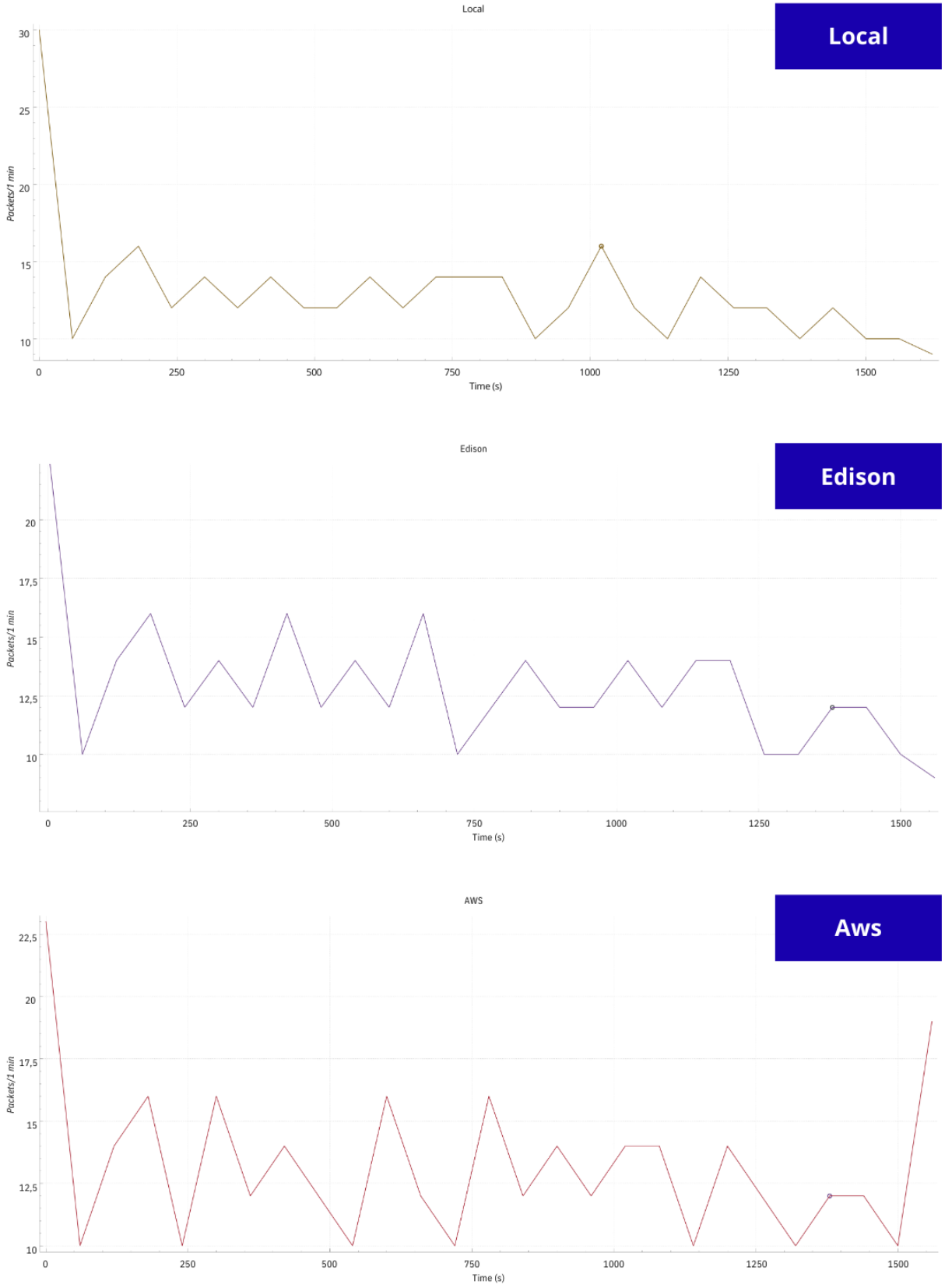
Fonte: Próprio autor (2026).

No ambiente AWS, o processo do Mosquitto registrou 0,0% de uso de CPU e 0,9% de memória, com memória total da instância de 945 MB. No Intel Edison, o *broker* não apareceu entre os processos de maior consumo: os processos dominantes foram os serviços Node.js do sistema operacional do próprio dispositivo (12%, 11% e 8% de VSZ), enquanto o Mosquitto operou com consumo de memória inferior a 1%; a carga média do Edison foi de 0,02 a 0,05, indicando ociosidade do sistema fora das demandas dos processos nativos. No ambiente local, o Mosquitto registrou 0,0% de CPU e 0,1% de memória, com carga média da máquina em torno de 2,97, reflexo dos processos de simulação do CoppeliaSim e dos *scripts* Python em execução simultânea. Em todos os ambientes, o Mosquitto não representou um gargalo de recursos, operando de forma estável sem picos de CPU associados ao volume de mensagens transmitidas.

O protocolo MQTT operou como *middleware* de comunicação entre os agentes, estabelecendo um formato único para a troca de mensagens. Essa padronização permitiu a comunicação entre os diferentes componentes do sistema, independente do ambiente de execução. O *broker* foi configurado com autenticação por usuário e senha, restringindo o acesso aos clientes autorizados. Essa configuração foi aplicada no ambiente experimental e permitiu o controle das conexões ao serviço de mensagens.

A Figura 18 apresenta o volume de pacotes transmitidos por minuto nos três ambientes avaliados ao longo do período de execução.

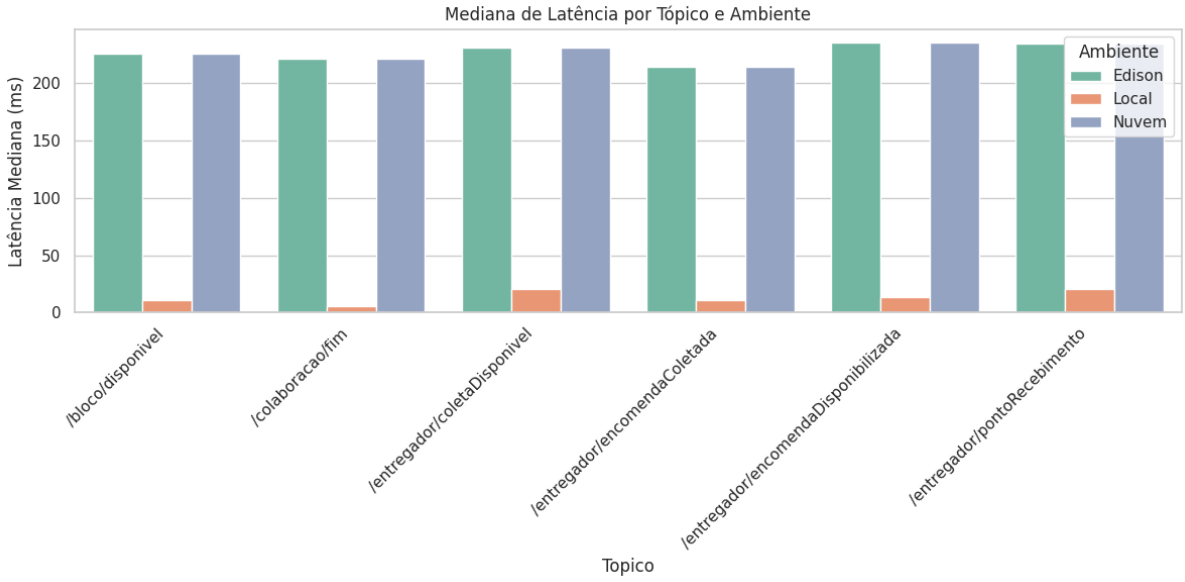
Figura 18 – Pacotes de dados por minuto nos ambientes AWS, Edison e local.



Fonte: Próprio autor (2026).

A latência mediana e o *jitter* foram calculados para os seis tópicos MQTT utilizados durante o ciclo operacional, nas três configurações de hospedagem do *broker*. A Figura 19 apresenta a mediana das latências registradas por tópico e ambiente. O ambiente local apresentou medianas inferiores a 25 ms em todos os tópicos, caracterizando comunicação quase imediata. Nos ambientes em nuvem (Amazon EC2) e em dispositivo de borda (Intel Edison), as latências foram consistentemente superiores a 200 ms, refletindo o impacto do roteamento externo sobre o tempo de entrega das mensagens. Apesar dessas diferenças, todos os eventos foram transmitidos e processados corretamente nos três ambientes, sem comprometer a execução do fluxo operacional.

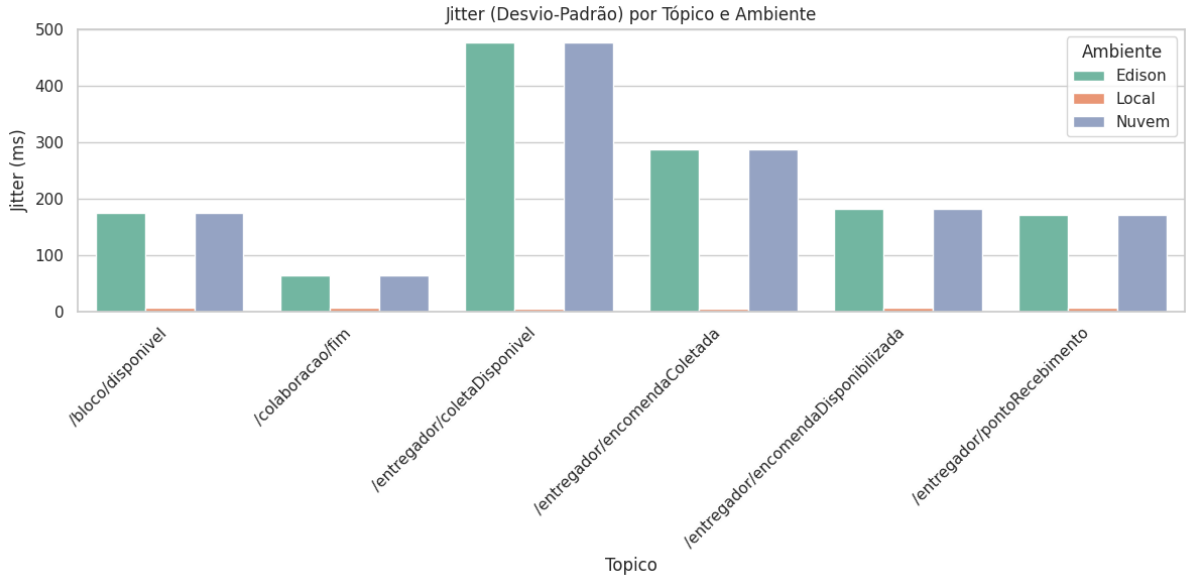
Figura 19 – Latência mediana por tópico MQTT nos ambientes local, AWS e Edison.



Fonte: Próprio autor (2026).

A Figura 20 apresenta o *jitter* calculado como desvio padrão das latências registradas por tópico. No ambiente local, o *jitter* permaneceu próximo de zero, indicando transmissão estável e regular. Nos ambientes externo e em borda, os valores variaram entre 50 ms e até 480 ms, com maior instabilidade registrada no tópico `/entregador/coletaDisponivel`. Apesar das oscilações nos ambientes externos, não foram registradas falhas de entrega ou perda de mensagens em nenhuma das configurações avaliadas.

Figura 20 – Jitter por tópico MQTT nos ambientes local, AWS e Edison.



Fonte: Próprio autor (2026).

Durante as 15 execuções realizadas, totalizando 150 ciclos operacionais completos, todos os ciclos foram concluídos corretamente, resultando em taxa de sucesso de 100%. O fluxo colaborativo de detecção, coleta, transporte e armazenamento ocorreu de forma ininterrupta no CoppeliaSim EDU, sem inconsistências operacionais entre os agentes. Esses resultados confirmam a confiabilidade e a estabilidade operacional da arquitetura nos três ambientes de hospedagem avaliados. As métricas completas de latência e *jitter* por tópico estão consolidadas no Apêndice A, e os registros brutos que originaram esses indicadores estão disponíveis no Apêndice D.

4.2 TREINAMENTO PERCEPTIVO DOS MODELOS

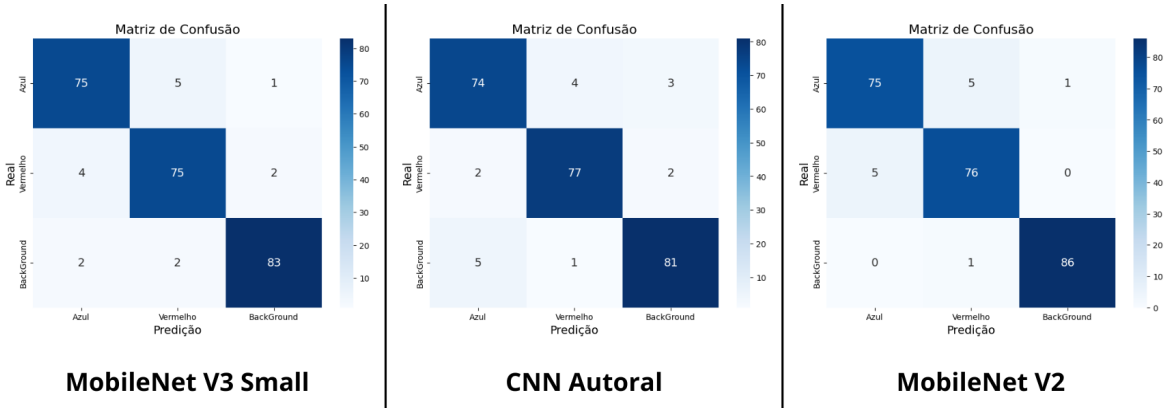
O treinamento supervisionado foi realizado utilizando as arquiteturas MobileNetV2, MobileNetV3 Small e uma rede convolucional autoral, com o objetivo de classificar imagens nas classes Azul, Vermelho e *Background*. As arquiteturas MobileNetV2 e MobileNetV3 Small foram inicializadas com pesos pré-treinados no ImageNet e adaptadas à tarefa por meio da substituição da camada de classificação original por uma nova camada densa com três neurônios e função de ativação *softmax*. Inicialmente, as camadas convolucionais dessas arquiteturas permaneceram congeladas, sendo treinadas apenas as camadas finais adicionadas. Após essa etapa, as últimas camadas convolucionais foram liberadas para treinamento, permitindo o ajuste fino dos parâmetros com taxa de aprendizado reduzida.

O treinamento foi conduzido em duas fases. Na primeira fase, o modelo foi treinado com as camadas convolucionais congeladas, permitindo o ajuste das camadas finais responsáveis

pela classificação. Na segunda fase, parte das camadas convolucionais foi descongelada, possibilitando o refinamento das representações internas com base nas características do conjunto de dados. Esse procedimento foi aplicado às arquiteturas MobileNetV2 e MobileNetV3 Small. A rede convolucional autoral foi treinada desde o início com todos os parâmetros livres, utilizando camadas convolucionais, camadas de redução dimensional e uma camada de saída com três neurônios.

A Figura 21 apresenta as matrizes de confusão obtidas no conjunto de teste para os três modelos avaliados. A concentração de acertos na diagonal principal foi alta em todas as arquiteturas, indicando que as classes previstas corresponderam às classes reais na maior parte dos casos. Os erros de classificação ocorreram principalmente entre a classe *Background* e as demais classes, sem comprometer o desempenho global dos modelos.

Figura 21 – Matrizes de confusão dos modelos avaliados.



Fonte: Próprio autor (2026).

As métricas quantitativas obtidas no conjunto de teste estão apresentadas na Tabela 3. A arquitetura MobileNetV2 apresentou acurácia de 95,18%, com valores equilibrados de precisão, revocação e F1-*score*. A MobileNetV3 Small apresentou acurácia de 93,57%, mantendo desempenho próximo ao da MobileNetV2. A rede convolucional autoral apresentou acurácia de 93,17%, com valores consistentes nas métricas avaliadas.

Tabela 3 – Métricas de desempenho dos modelos de classificação no conjunto de teste

Modelo	**Acurácia**	**Macro Precisão**	**Macro Revocação**	**Macro F1-Score**
MobileNet V2	0,9518	0,9509	0,9509	0,9509
MobileNet V3 Small	0,9357	0,9352	0,9353	0,9352
CNN autoral	0,9317	0,9315	0,9317	0,9316

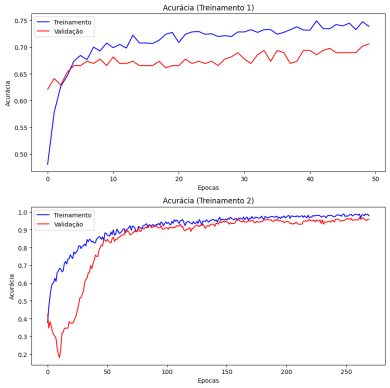
Fonte: elaborada pelo autor (2026).

Os valores obtidos indicam desempenho semelhante entre as arquiteturas, com diferença inferior a 3% entre o melhor e o pior resultado. As três arquiteturas superaram 93% de acurácia, com capacidade de classificação adequada para a tarefa proposta. As métricas quantitativas confirmam os padrões observados nas matrizes de confusão.

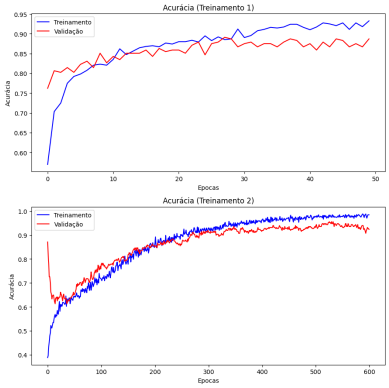
Durante o treinamento, foram registrados os valores de acurácia e erro ao longo das épocas. A Figura 22 apresenta as curvas de acurácia das três arquiteturas. A acurácia aumentou progressivamente nas fases iniciais do treinamento e se estabilizou nas épocas finais, indicando convergência dos parâmetros. A MobileNetV2 atingiu o maior valor final, seguida pela MobileNetV3 Small e pela CNN autoral, resultado consistente com as métricas do conjunto de teste.

A Figura 23 apresenta as curvas de erro correspondentes ao treinamento. Em todas as arquiteturas, o erro reduziu nas fases iniciais e se estabilizou nas épocas finais, com comportamento coerente ao aumento da acurácia. A estabilização indica convergência dos parâmetros sem degradação de desempenho nas épocas finais.

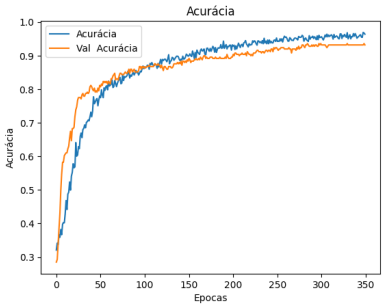
Figura 22 – Curvas de acurácia durante o treinamento dos modelos.



(a) MobileNetV2



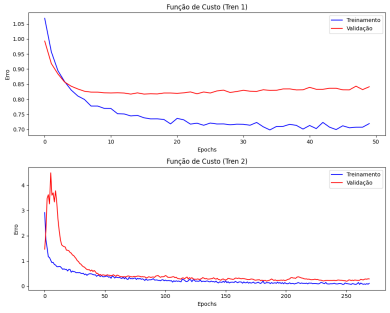
(b) MobileNetV3 Small



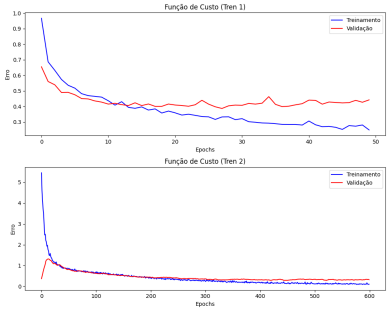
(c) CNN autoral

Fonte: Elaborado pelo autor (2026).

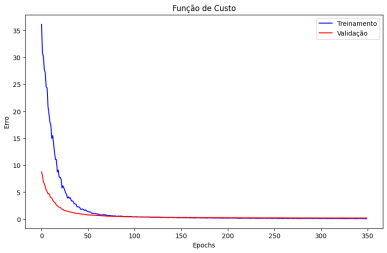
Figura 23 – Curvas de erro durante o treinamento dos modelos.



(a) MobileNetV2



(b) MobileNetV3 Small



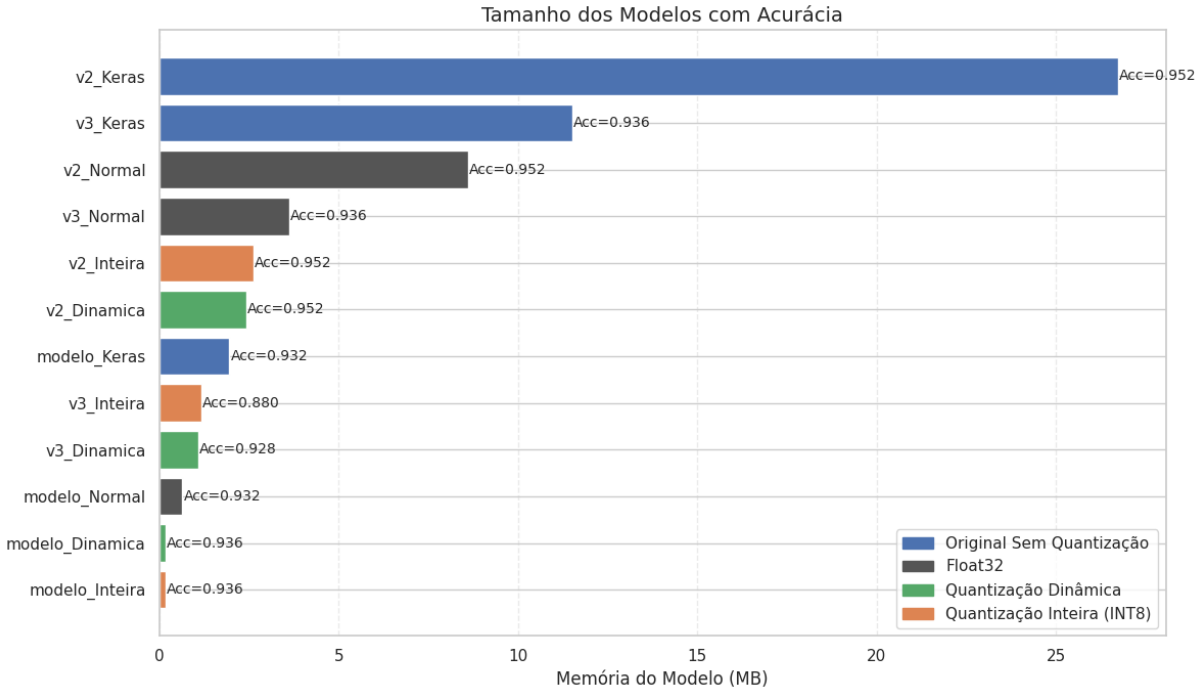
(c) CNN autoral

Fonte: Elaborado pelo autor (2026).

4.3 IMPACTO DA CONVERSÃO E QUANTIZAÇÃO NOS MODELOS

A conversão do formato Keras para *TensorFlow Lite* produziu reduções expressivas no tamanho dos arquivos em todas as arquiteturas. A Figura 24 compara os modelos originais com suas versões convertidas, nas variantes sem quantização, com quantização dinâmica e com quantização inteira. Os modelos originais, em ponto flutuante de 32 bits, apresentaram o maior tamanho. A redução foi progressiva com o nível de quantização aplicada, padrão consistente nas três arquiteturas e diretamente associado ao formato de representação numérica dos parâmetros.

Figura 24 – Comparação do tamanho dos modelos originais e das versões TensorFlow Lite.



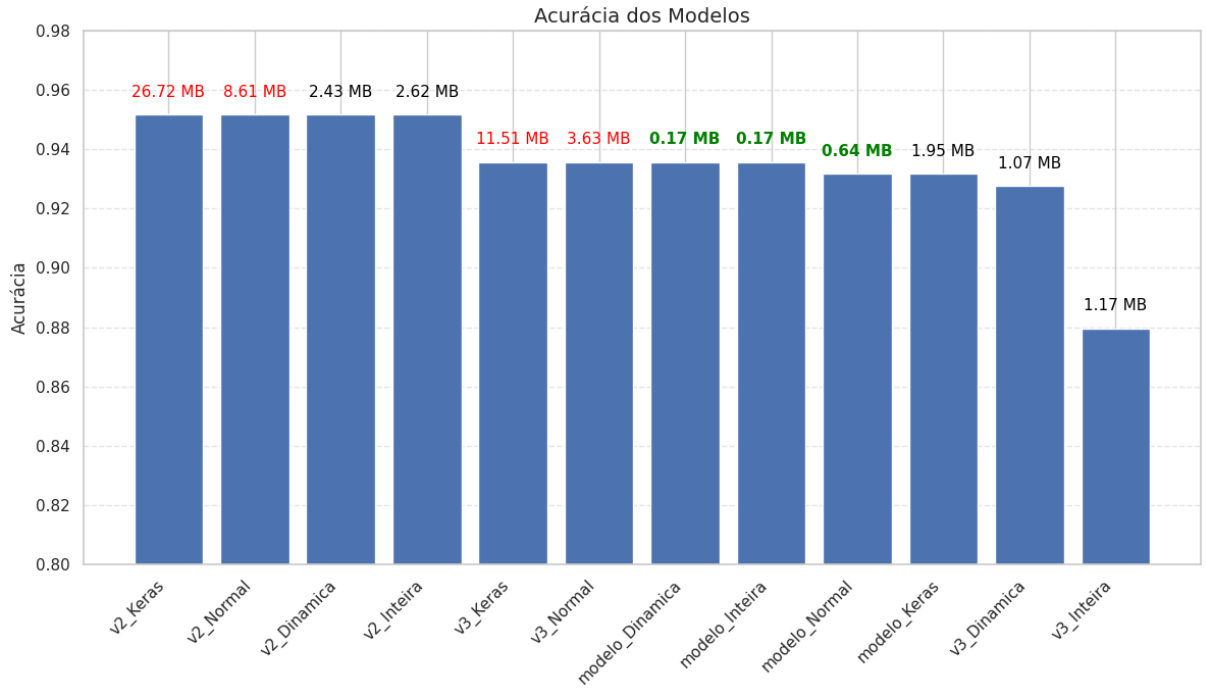
Fonte: Próprio autor (2026).

A quantização dinâmica produziu redução intermediária, enquanto a quantização inteira resultou nas menores representações estruturais. A redução observada está associada à conversão dos pesos e parâmetros internos para representações inteiras de 8 bits, diminuindo a quantidade de dados armazenados no modelo. Essa alteração estrutural não modificou a arquitetura das redes, mantendo o número de camadas e a organização dos parâmetros. A CNN autoral apresentou o menor tamanho absoluto após a conversão, enquanto a MobileNetV2 e a MobileNetV3 Small apresentaram reduções proporcionais semelhantes, embora com tamanhos finais superiores devido à maior complexidade estrutural.

A relação entre o tamanho dos modelos e a acurácia obtida após a conversão está apresentada na Figura 25. A MobileNetV2 manteve o maior valor de acurácia entre os modelos convertidos, mesmo após a aplicação da quantização inteira. A MobileNetV3 Small apresentou

redução de tamanho acompanhada por diminuição de acurácia, mantendo desempenho funcional para a tarefa de classificação. A CNN autoral apresentou o menor tamanho entre as arquiteturas avaliadas e manteve acurácia próxima à obtida no modelo original, caracterizando uma relação favorável entre tamanho e desempenho.

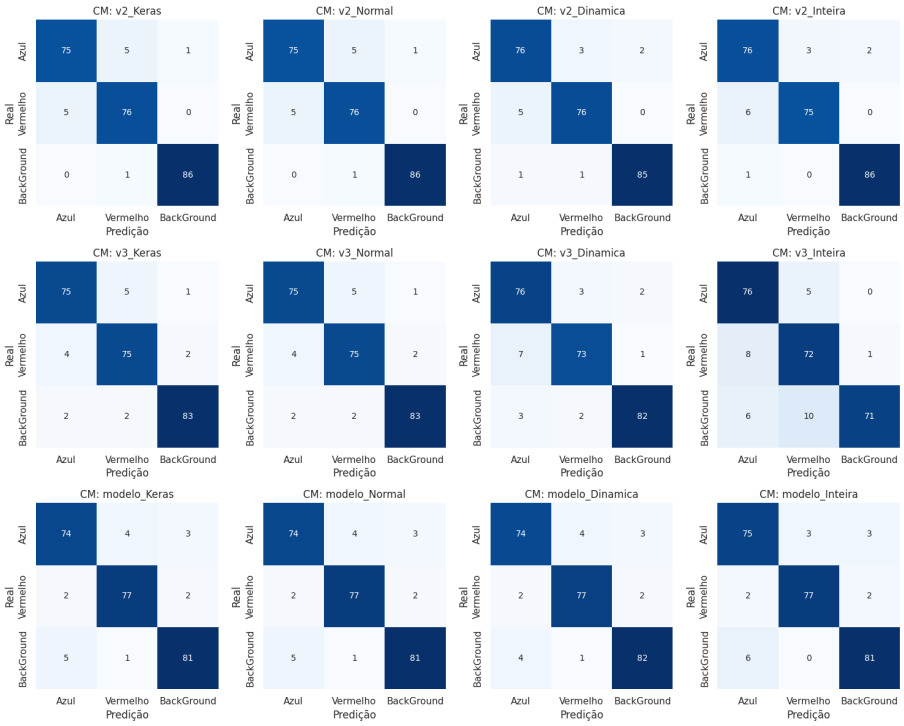
Figura 25 – Relação entre tamanho do modelo TFLite e acurácia após conversão.



Fonte: Próprio autor (2026).

A Figura 26 apresenta as matrizes de confusão obtidas após a conversão dos modelos para o formato *TensorFlow Lite*. A distribuição dos acertos permaneceu concentrada na diagonal principal, indicando correspondência entre as classes previstas e as classes reais. As diferenças em relação aos modelos originais ocorreram principalmente na classe *Background*, com aumento discreto de classificações incorretas em algumas arquiteturas. As classes Azul e Vermelho mantiveram maior taxa de acerto, sem alteração significativa após a conversão.

Figura 26 – Matrizes de confusão dos modelos convertidos para o formato TensorFlow Lite.



Fonte: Próprio autor (2026).

A conversão produziu versões estruturalmente menores e compatíveis com restrições de memória, preservando o funcionamento das arquiteturas treinadas. As versões quantizadas apresentaram redução significativa no tamanho dos arquivos e mantiveram capacidade de classificação compatível com a tarefa proposta.

4.4 VIABILIDADE DE EMBARQUE NO ESP32

A viabilidade de execução dos modelos no dispositivo ESP32-S3-WROOM-1-N16R8 foi determinada a partir da análise do tamanho dos modelos convertidos e do consumo de memória necessário durante a execução. A Tabela 4 apresenta os valores de acurácia, ocupação de memória Flash e consumo da arena de execução para as arquiteturas avaliadas. A quantização inteira resultou em reduções expressivas no tamanho dos modelos, reduzindo a MobileNetV2 de 26,72 MB para 2,62 MB e a MobileNetV3 Small de 11,51 MB para 1,17 MB. A CNN autoral apresentou redução de 1,95 MB para 0,16 MB, representando a menor ocupação de memória entre os modelos avaliados.

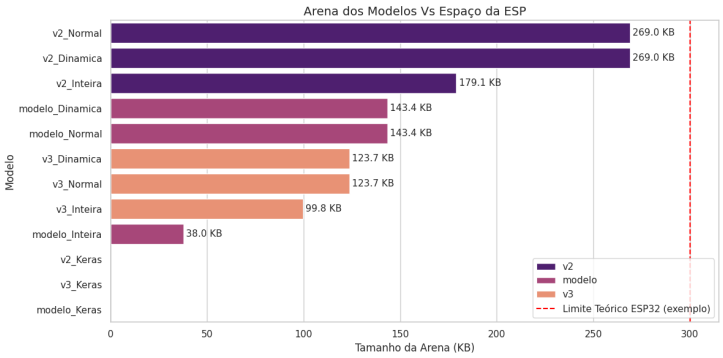
Tabela 4 – Comparação entre modelos quanto ao uso de memória e acurácia

Arquitetura	Tipo	Acurácia	Flash (MB)	Redução (%)	Arena (KB)
MobileNet V2	Keras	0,9518	26,72	—	—
MobileNet V2	INT8	0,9518	2,62	90,18	179,1
MobileNet V3 Small	Keras	0,9357	11,51	—	—
MobileNet V3 Small	INT8	0,8795	1,17	89,79	99,8
CNN autoral	Keras	0,9317	1,95	—	—
CNN autoral	INT8	0,9357	0,16	91,42	37,9

Fonte: elaborada pelo autor (2026).

A Figura 27 apresenta o consumo estimado de arena para cada arquitetura, medido em ambiente Linux com a biblioteca TFLite padrão como passo inicial antes do embarque. Os valores registrados foram: MobileNetV2 com 179,1 KB, MobileNetV3 Small com 99,8 KB e CNN autoral com 37,9 KB. Esses dados serviram como referência inicial para avaliar a viabilidade de execução no dispositivo.

Figura 27 – Estimativa inicial do consumo de arena em relação ao limite prático do ESP32-S3.



Fonte: Próprio autor (2026).

O ESP32-S3 dispõe de aproximadamente 377 KB de SRAM disponível ao interpretador após a inicialização do *firmware* com Wi-Fi e MQTT ativos, valor obtido via diagnóstico direto no *hardware*. Com base nessa estimativa, todos os modelos pareciam viáveis. Os testes de execução real no dispositivo, contudo, revelaram discrepâncias significativas entre a estimativa e o consumo efetivo do TFLite Micro, conforme detalhado na sequência.

A avaliação da viabilidade de embarque revelou três obstáculos técnicos que exigiram intervenções específicas. O primeiro foi a falha de alocação da arena de tensores para a MobileNetV2: embora a medição realizada no *notebook* apontasse uma arena de 179,1 KB, o interpretador *TensorFlow Lite Micro* no ESP32 exigiu 558.504 bytes (545 KB), valor superior aos aproximados 377 KB de SRAM disponíveis após a inicialização do sistema. Para contornar esse

problema, a RAM externa (PSRAM) foi ativada no `menuconfig` em modo Octal a 80 MHz, e a alocação da arena foi redirecionada para essa região por meio da função `heap_caps_malloc` com a *flag* `MALLOC_CAP_SPIRAM`.

O segundo obstáculo foi o estouro do limite de memória Flash durante a compilação. O binário do projeto com a MobileNetV2 embutida atingiu 3,76 MB, ultrapassando o limite da partição padrão do ESP-IDF. A solução foi a criação de um arquivo `partitions.csv` personalizado reservando 8 MB para a partição `factory`, com ajuste simultâneo da Flash de 2 MB para 16 MB no `menuconfig`.

O terceiro obstáculo foi a discrepância entre os valores de arena medidos fora do *hardware* e os valores reais exigidos pelo ESP32. A Tabela 5 apresenta essa comparação para as três arquiteturas avaliadas, obtida por meio da função `arena_used_bytes()` executada diretamente no dispositivo.

Tabela 5 – Comparação entre consumo de arena estimado no notebook e medido no ESP32

Arquitetura	**Arena no Notebook (KB)**	**Arena no ESP32 (KB)**	**Diferença (%)**
MobileNet V2 INT8	179,1	545,4	+204,5
MobileNet V3 Small INT8	99,8	166,3	+66,6
CNN Autoral INT8	37,9	38,4	+1,3

Fonte: elaborada pelo autor (2026).

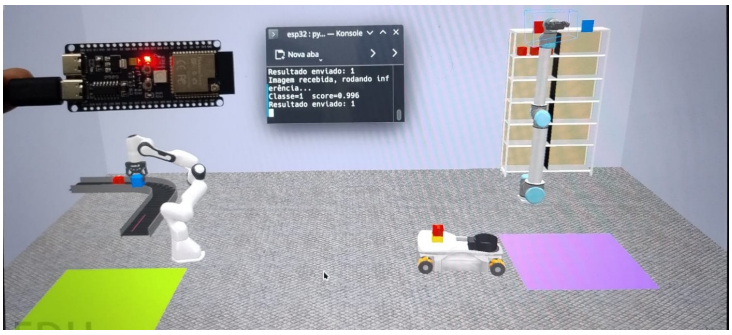
Os dados mostram que as estimativas externas subestimaram os requisitos reais das arquiteturas MobileNet. Essa discrepância ocorre porque a biblioteca *TensorFlow Lite Micro* (TFLm) utilizada no microcontrolador aloca *buffers* intermediários adicionais e exige alocação estática extra para lidar com as restrições arquiteturais e de tipos de dados durante o fluxo de invocação da inferência, fatores que não são refletidos nas estimativas geradas pelo *TensorFlow Lite* padrão no ambiente de desenvolvimento. A CNN autoral foi a única com medições convergentes entre os dois ambientes (diferença de 1,3%), confirmando sua adequação ao *hardware* embarcado. Mesmo após a resolução dos problemas de memória, a MobileNetV2 produziu classificações incorretas para imagens 32×32 recebidas via MQTT. O modelo foi descartado em favor da CNN autoral, cujos 38,4 KB de arena na SRAM interna dispensaram o uso de PSRAM.

4.5 RESULTADOS DA INTEGRAÇÃO COM INFERÊNCIA EMBARCADA

A etapa de integração avaliou o comportamento do sistema com o processamento de classificação executado no dispositivo embarcado ESP32-S3-WROOM-1-N16R8. A Figura 28 apresenta o sistema em operação, incluindo o dispositivo físico energizado, o terminal com os registros gerados pelo *firmware* e o ambiente de simulação com os agentes robóticos em execução. Os registros do terminal evidenciam confiança de 0,996 (99,6%) na classificação

realizada pelo modelo CNN autoral, confirmando a operação correta do interpretador *TensorFlow Lite Micro* no *hardware*.

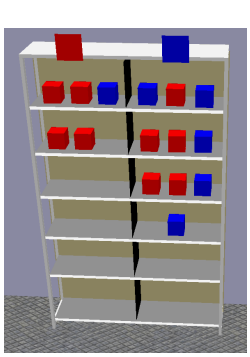
Figura 28 – Sistema IoRT em operação: ESP32 físico, terminal de logs e cenário simulado.



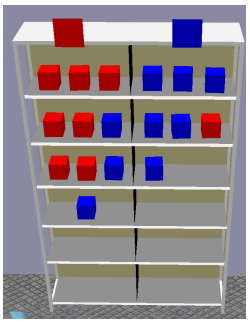
Fonte: Próprio autor (2026).

O comportamento do ciclo operacional foi avaliado nos três modelos candidatos à integração: MobileNetV2, MobileNetV3 Small e CNN autoral. A Figura 29 apresenta o estado da estante ao final de uma sequência de operações para cada arquitetura testada. A MobileNetV2 produziu classificações incorretas de forma sistemática, resultando em cubos azuis e vermelhos distribuídos de forma desordenada na estante. A MobileNetV3 Small apresentou comportamento semelhante, com erros de classificação que impediram o posicionamento correto dos objetos. A CNN autoral classificou corretamente todos os objetos avaliados, com cubos vermelhos e azuis posicionados em suas respectivas posições, resultando em organização completa e sem erros ao final do ciclo.

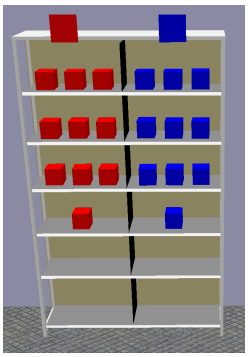
Figura 29 – Estado da estante após execução



(a) MobileNetV2



(b) MobileNetV3 Small



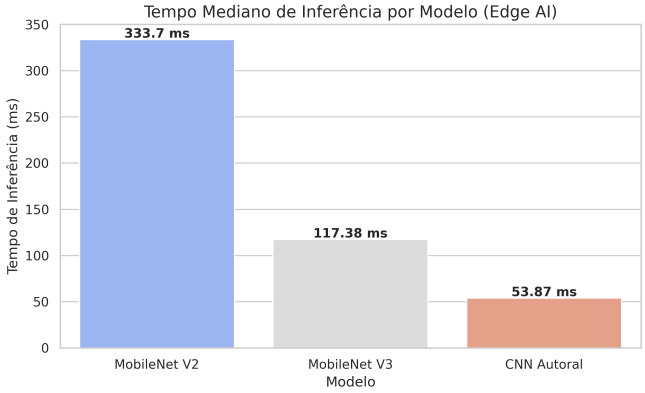
(c) CNN Autoral

Fonte: Elaborado pelo autor (2026).

O tempo de inferência medido no ESP32 variou significativamente entre as arquiteturas. A Figura 30 apresenta os valores registrados para cada modelo. A CNN autoral apresentou o menor tempo de inferência, com 53,87 ms por classificação, seguida pela MobileNetV3 Small

com 117,38 ms e pela MobileNetV2 com 333,7 ms. Esses valores refletem diretamente a complexidade computacional de cada arquitetura e sua adequação às restrições de processamento do microcontrolador. Os registros de inferência detalhados por nível de confiança e execução estão disponíveis no Apêndice C.

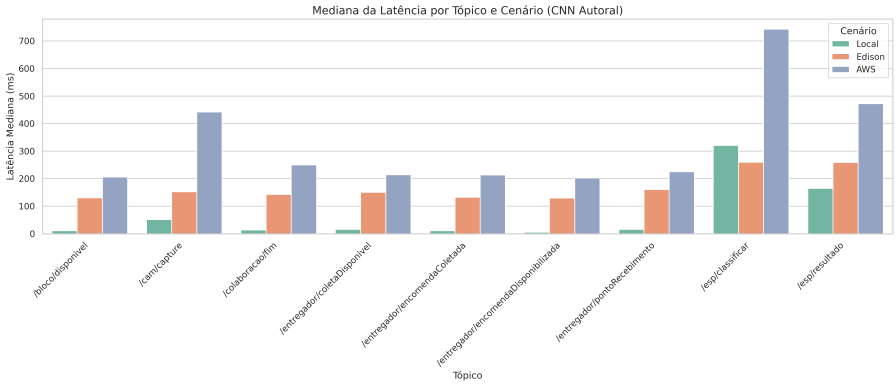
Figura 30 – Tempo de inferência no ESP32 para cada arquitetura.



Fonte: Próprio autor (2026).

A integração da inferência embarcada ao fluxo operacional introduziu dois novos tópicos MQTT: o tópico de envio da imagem ao ESP32 e o tópico de recebimento do resultado da classificação. A Figura 31 apresenta a latência mediana dos oito tópicos do sistema com a inferência ativa. O tópico `/esp/classificar` apresentou o maior valor de latência, com aproximadamente 320 ms no ambiente local e 740 ms no ambiente em nuvem (AWS), refletindo o tempo de processamento da inferência somado ao tempo de transmissão da imagem. Os demais tópicos mantiveram latências compatíveis com os resultados registrados na etapa sem inferência embarcada.

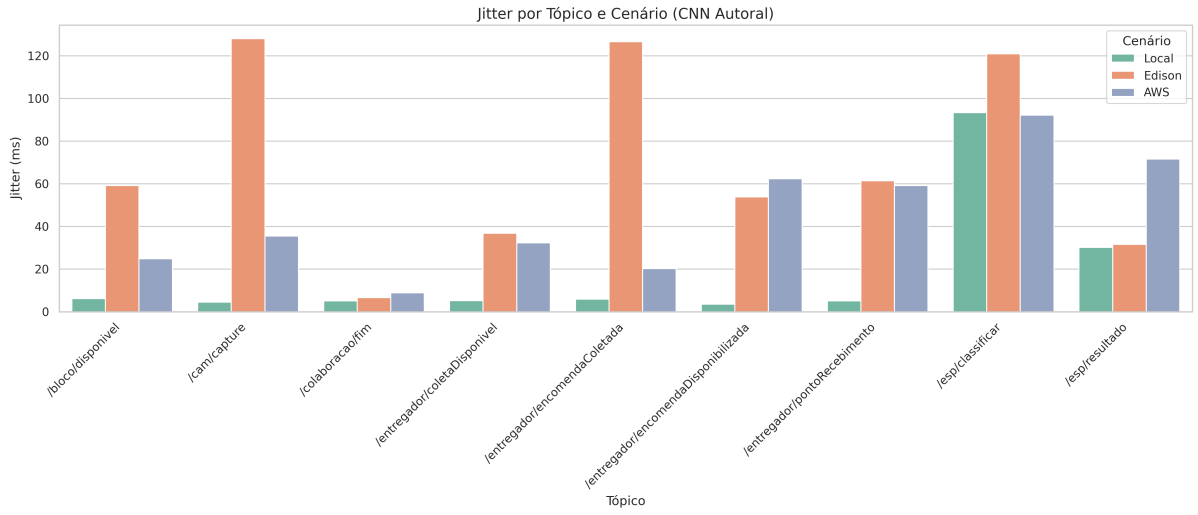
Figura 31 – Latência mediana por tópico com inferência embarcada ativa nos ambientes.



Fonte: Próprio autor (2026).

A Figura 32 apresenta o *jitter* registrado durante a execução com inferência embarcada ativa. Os valores variaram entre 10 ms no ambiente local e 120 ms nos ambientes Edison e AWS, resultado inferior aos picos de até 480 ms registrados na etapa sem inferência. Esse comportamento se explica parcialmente pelo fato de que o tempo de inferência do ESP32 (53,87 ms) introduz um intervalo fixo entre as mensagens do ciclo de classificação, regularizando o ritmo de publicação para os tópicos subsequentes ao `/esp/classificar`. A variabilidade temporal passou a ser dominada pelo tempo de processamento no microcontrolador nos ambientes com menor latência de rede, enquanto nos ambientes externos o roteamento externo continuou contribuindo com a maior parte do *jitter* nos demais tópicos. O fato de o *jitter* com inferência ativa ter ficado abaixo do registrado na etapa sem inferência indica que a adição do dispositivo embarcado ao fluxo não degradou a previsibilidade temporal do sistema.

Figura 32 – Jitter por tópico com inferência embarcada ativa nos ambientes local, AWS e Edison.



Fonte: Próprio autor (2026).

As cinco execuções realizadas com a CNN autoral, cada uma composta por 40 operações de classificação, foram concluídas integralmente nos três ambientes, resultando em taxa de sucesso de 100% no fluxo com inferência embarcada. O ciclo completo incluiu captura de imagem pelo sensor de visão, transmissão da imagem ao ESP32 via MQTT, classificação no dispositivo, publicação do resultado e atuação do manipulador UR10 para posicionar o objeto na prateleira correta. Nenhuma dessas etapas gerou falha ou inconsistência operacional ao longo das 200 operações registradas. O acréscimo de latência introduzido pela inferência embarcada, de 320 ms a 740 ms dependendo do ambiente, foi absorvido pelo ritmo de execução da simulação sem causar interrupções no fluxo colaborativo. As métricas consolidadas de latência e *jitter* com inferência ativa estão no Apêndice B, e os registros brutos correspondentes no Apêndice E.

5 CONCLUSÃO

Este trabalho propôs e avaliou uma arquitetura IoRT multiagente de quatro camadas que integra inferência de visão computacional executada localmente em um microcontrolador ESP32-S3-WROOM-1-N16R8 ao fluxo de comunicação MQTT de um sistema robótico colaborativo simulado. A camada de comunicação operou com taxa de sucesso de 100% nas 150 execuções sem inferência embarcada realizadas nos três ambientes avaliados, com latência mediana inferior a 25 ms no ambiente local e superior a 200 ms nos ambientes externo e em borda, sem comprometer a conclusão dos ciclos. As três arquiteturas de redes neurais treinadas, MobileNetV2 (95,18%), MobileNetV3 Small (93,57%) e CNN autoral (93,17%), atingiram acurácia superior a 93% no conjunto de teste, e a conversão para *TensorFlow Lite* com quantização inteira reduziu o tamanho dos modelos em mais de 89% em todos os casos.

O principal desafio técnico identificado foi a discrepância entre as estimativas de consumo de arena de memória calculadas fora do dispositivo e os valores reais exigidos pelo interpretador *TensorFlow Lite Micro* no ESP32-S3. A MobileNetV2 necessitou de 545,4 KB de arena contra os 179,1 KB estimados no *notebook*, e a MobileNetV3 Small exigiu 166,3 KB contra os 99,8 KB estimados. Apenas a CNN autoral apresentou convergência entre os dois ambientes, com diferença de 1,3%. Esse resultado revela uma lacuna concreta no ciclo de desenvolvimento de sistemas Edge AI: as ferramentas de perfilamento externas ao *hardware* subestimam sistematicamente os requisitos reais do TFLite Micro em microcontroladores, aspecto diretamente relevante para qualquer projeto que selecione modelos com base em estimativas feitas fora do dispositivo alvo, e que raramente aparece discutido de forma explícita na literatura de sistemas embarcados. A MobileNetV2 foi descartada após produzir classificações incorretas no *hardware* real, mesmo com PSRAM ativa, e a CNN autoral foi adotada como modelo definitivo.

Com a CNN autoral integrada ao *firmware*, o sistema realizou 200 operações de classificação em cinco execuções, com taxa de sucesso de 100% nos três ambientes, tempo de inferência de 53,87 ms e confiança média de 99,6% nas classificações. A estante foi organizada corretamente ao final de cada ciclo, com os agentes robóticos coordenados exclusivamente pelas mensagens MQTT geradas a partir da classificação embarcada. O tópico de classificação adicionou latência de 320 ms no ambiente local e 740 ms no ambiente em nuvem, reflexo do tempo de processamento no microcontrolador, mas sem impedir a conclusão das tarefas.

Os resultados colocam em perspectiva um ponto relevante sobre a adoção de modelos de transferência de aprendizado em *hardware* embarcado. A MobileNetV2 obteve a maior acurácia entre as três arquiteturas no conjunto de teste e seria a escolha natural em um cenário de implantação em servidor. No ESP32-S3, o mesmo modelo consumiu mais de 500 KB de arena e ainda assim produziu classificações incorretas para imagens 32×32, enquanto a CNN autoral, com acurácia 2 pontos percentuais inferior, operou dentro da SRAM interna com 38 KB de arena e classificações corretas no *hardware*. Esse contraste evidencia que, para sistemas embarcados

com restrições severas de memória, a escolha da arquitetura não pode ser baseada apenas em *benchmarks* realizados fora do dispositivo alvo. Conclui-se, portanto, que a viabilidade do Edge AI no microcontrolador ESP32 depende diretamente do consumo real de arena e da escolha iterativa de arquiteturas compactas projetadas para ambientes com memória extremamente restrita.

5.1 TRABALHOS FUTUROS

O próximo passo natural é a transição do sistema para robôs físicos, verificando se a arquitetura se sustenta diante de variáveis ausentes na simulação: latência real dos atuadores, ruído dos sensores e variabilidade mecânica dos manipuladores. No âmbito da visão computacional, a substituição da CNN por um modelo de linguagem visual de pequeno porte (*Small Vision Language Model*: SmolVLM, Moondream ou similares) permitiria eliminar a etapa de classificação por categorias fixas, abrindo espaço para decisões baseadas em descrição visual livre. Nessa mesma direção, a coordenação do sistema por comandos em linguagem natural, em vez da sincronização por eventos fixos via MQTT, é uma extensão relevante, já que modelos como SmolLM e variantes quantizadas em INT4 já operam em *hardware* com restrições de memória comparáveis ao ESP32-S3.

Dois problemas técnicos identificados neste trabalho merecem investigação específica. O primeiro é a queda de desempenho das redes MobileNetV2 e MobileNetV3 Small no ambiente embarcado: ambas apresentaram erros sistemáticos de classificação no ESP32, mesmo após a resolução das restrições de memória, comportamento que não se reproduziu no ambiente de desenvolvimento. Compreender as causas dessa degradação (se relacionadas à quantização inteira, ao tamanho da imagem de entrada de 32×32 ou à forma como o interpretador TFLite Micro lida com camadas específicas dessas arquiteturas) é necessário antes de qualquer decisão de descarte para aplicações embarcadas. O segundo problema é que apenas a quantização inteira foi avaliada como técnica de compressão. Técnicas como *network pruning* e *knowledge distillation* ainda não foram testadas no contexto deste sistema, e estudar se elas produzem modelos mais adequados ao embarque no ESP32, seja em consumo de arena, seja em acurácia no *hardware* real, é uma lacuna direta deste trabalho.

Na camada de comunicação, executar o *broker* MQTT diretamente no ESP32-S3 ou em um dispositivo de borda dedicado eliminaria a dependência de servidores externos e reduziria a latência nos ciclos com inferência ativa. A adição de TLS ao canal MQTT é requisito para aplicações industriais reais, e seu impacto no consumo de memória e no *throughput* do *firmware* precisa ser quantificado para dispositivos com SRAM restrita.

REFERÊNCIAS BIBLIOGRÁFICAS

ABUGHAZALA, M. *et al.* An architecture framework for architecting IoT applications: From design to deployment. **J. Syst. Softw.**, v. 234, n. 112728, p. 112728, 2026. Citado na página 22.

AFANASYEV, I. *et al.* Towards the internet of robotic things: Analysis, architecture, components and challenges. In: **2019 12th International Conference on Developments in eSystems Engineering (DeSE)**. [S.l.]: IEEE, 2019. Citado 4 vezes nas páginas 15, 17, 18 e 23.

ASTUDILLO, T.; MISSAEL, A. **Estudio, control e implementación de sistemas robóticos avanzados: aplicación de un algoritmo basado en consenso para un sistema multi-agente robótico simulado en CoppeliaSim y comandado desde MATLAB**. Dissertação (Mestrado) — Escuela Politécnica Nacional, Quito, 2022. Citado 2 vezes nas páginas 17 e 28.

AZZEDIN, F.; ALHAZMI, T. Secure data distribution architecture in IoT using MQTT. **Appl. Sci. (Basel)**, v. 13, n. 4, p. 2515, 2023. Citado na página 26.

BADIA, J. M. *et al.* Reliability of vision transformers and CNNs on edge AI systems under neutron radiation. **IEEE Trans. Nucl. Sci.**, v. 72, n. 8, p. 2706–2716, 2025. Citado 2 vezes nas páginas 25 e 27.

BATTH, R. S.; NAYYAR, A.; NAGPAL, A. Internet of robotic things: Driving intelligent robotics of future - concept, architecture, applications and technologies. In: **2018 4th International Conference on Computing Sciences (ICCS)**. [S.l.]: IEEE, 2018. Citado 4 vezes nas páginas 12, 13, 14 e 16.

BOGAERTS, B. *et al.* Connecting the CoppeliaSim robotics simulator to virtual reality. **SoftwareX**, v. 11, n. 100426, p. 100426, 2020. Citado 2 vezes nas páginas 28 e 29.

CRUZ, N.; LOBOS-TSUNEKAWA, K.; SOLAR, J. Ruiz-del. Using convolutional neural networks in robots with limited computational resources: Detecting NAO robots while playing soccer. In: **RoboCup 2017: Robot World Cup XXI**. Cham: Springer International Publishing, 2018. p. 19–30. Citado na página 24.

DALENOGARE, L. S. *et al.* The expected contribution of industry 4.0 technologies for industrial performance. **Int. J. Prod. Econ.**, v. 204, p. 383–394, 2018. Citado na página 12.

DAVE, M.; DOSHI, J.; AROLKAR, H. MQTT- CoAP interconnector: IoT interoperability solution for application layer protocols. In: **2020 Fourth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC)**. [S.l.]: IEEE, 2020. Citado na página 26.

GHOBAKHLOO, M. *et al.* Behind the definition of industry 5.0: a systematic review of technologies, principles, components, and values. **J. Ind. Prod. Eng.**, p. 1–16, 2023. Citado na página 12.

GIL, A. C. **Como Elaborar Projetos de Pesquisa**. 7. ed. São Paulo: GEN Atlas, 2022. Citado na página 30.

GONZALEZ, J. M. F.; CORONADO, E.; YAMANOBE, N. ROS-compatible robotics simulators for industry 4.0 and industry 5.0: A systematic review of trends and technologies. **Appl. Sci. (Basel)**, v. 15, n. 15, p. 8637, 2025. Citado na página 29.

Google LLC. **Post-training quantization**. 2024. Google for Developers – LiteRT. Disponível em: <https://ai.google.dev/edge/litert/conversion/tensorflow/quantization/post_training_quantization>. Acesso em: 20 fev. 2026. Citado na página 38.

KABIR, H.; THAM, M.-L.; CHANG, Y. C. Internet of robotic things for mobile robots: Concepts, technologies, challenges, applications, and future directions. **Digit. Commun. Netw.**, v. 9, n. 6, p. 1265–1290, 2023. Citado 3 vezes nas páginas 17, 18 e 22.

KOUTRAS, D. *et al.* Comparative analysis and implementation of HTTP3, MQTT, and CoAP for IoT applications. In: **Proceedings of the 28th Pan-Hellenic Conference on Progress in Computing and Informatics**. New York, NY, USA: ACM, 2024. p. 127–132. Citado na página 19.

KREJČÍ, J. *et al.* Internet of robotic things: Current technologies, challenges, applications, and future research topics. **Sensors (Basel)**, v. 25, n. 3, p. 765, 2025. Citado 3 vezes nas páginas 15, 23 e 24.

MARIAN, M. *et al.* A ROS-based control application for a robotic platform using the gazebo 3D simulator. In: **2020 21th International Carpathian Control Conference (ICCC)**. [S.l.]: IEEE, 2020. Citado na página 29.

MISHRA, B.; MISHRA, B.; KERTESZ, A. Stress-testing MQTT brokers: A comparative analysis of performance measurements. **Energies**, v. 14, n. 18, p. 5817, 2021. Citado 5 vezes nas páginas 19, 20, 21, 22 e 26.

MUSA, A. *et al.* Lightweight deep learning models for edge devices—a survey. **Int. J. Comput. Inf. Syst. Ind. Manag. Appl.**, v. 17, p. 18, 2025. Citado 2 vezes nas páginas 24 e 25.

NIETO, R. *et al.* Open-source ROS-based simulation for verification of FPGA robotics applications. **Microprocess. Microsyst.**, v. 113, n. 105143, p. 105143, 2025. Citado na página 28.

RAHMAN, H.; HUSSAIN, M. I. A comprehensive survey on semantic interoperability for internet of things: State-of-the-art and research challenges. **Trans. Emerg. Telecommun. Technol.**, v. 31, n. 12, 2020. Citado na página 22.

RAY, P. P. Internet of robotic things: Concept, technologies, and challenges. **IEEE Access**, v. 4, p. 9489–9500, 2016. Citado 6 vezes nas páginas 12, 13, 15, 16, 18 e 22.

ROMEO, L. *et al.* Internet of robotic things in smart domains: Applications and challenges. **Sensors (Basel)**, v. 20, n. 12, p. 3355, 2020. Citado 2 vezes nas páginas 13 e 26.

SORRI, K.; MUSTAFEE, N.; SEPPÄNEN, M. Revisiting IoT definitions: A framework towards comprehensive use. **Technol. Forecast. Soc. Change**, v. 179, n. 121623, p. 121623, 2022. Citado 4 vezes nas páginas 12, 13, 14 e 27.

TAKAYA, K. *et al.* Simulation environment for mobile robots testing using ROS and gazebo. In: **2016 20th International Conference on System Theory, Control and Computing (ICSTCC)**. [S.l.]: IEEE, 2016. Citado 3 vezes nas páginas 14, 28 e 29.

VERMESAN, O. *et al.* Internet of robotic things intelligent connectivity and platforms. **Front. Robot. AI**, v. 7, p. 104, 2020. Citado 5 vezes nas páginas 12, 13, 20, 23 e 26.

VILLA, D. *et al.* Internet of robotic things: Current technologies, applications, challenges and future directions. 2021. Citado 4 vezes nas páginas 16, 19, 20 e 23.

ZEGARRA, E. C.; ARANIBAR, D. B.; CARDINALE, Y. Iort-based middleware for heterogeneous multi-robot systems. **Journal of Sensor and Actuator Networks**, v. 13, n. 6, 2024. ISSN 2224-2708. Disponível em: <https://www.mdpi.com/2224-2708/13/6/87>. Citado 4 vezes nas páginas 18, 21, 22 e 27.

ZHANG, Y. *et al.* The implementation of CNN-based object detector on ARM embedded platforms. In: **2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress(DASC/PiCom/DataCom/CyberSciTech)**. [S.l.]: IEEE, 2018. Citado na página 24.

ŽIVIĆ, M.; NEMEC, D.; BOJOVIĆ, Ž. MQTT protocol in IoT environment: Comparison with CoAP and ZeroMQ protocols. In: **2023 31st Telecommunications Forum (TELFOR)**. [S.l.]: IEEE, 2023. p. 1–4. Citado na página 19.

Apêndices

APÊNDICE A – MÉTRICAS DE LATÊNCIA E JITTER POR TÓPICO MQTT — CENÁRIO SEM INFERÊNCIA EMBARCADA

Ambiente	Tópico	Mediana (ms)	Média (ms)	Jitter (ms)	P95 (ms)	Mín (ms)	Máx (ms)	Vol.
Local	/bloco/disponivel	10,79	13,23	6,98	26,26	0,64	26,49	60
	/colaboracao/fim	5,90	8,82	7,81	18,60	0,42	31,07	18
	/entregador/coletaDisponivel	20,75	20,48	6,36	31,26	5,64	36,44	60
	/entregador/encomendaColetada	10,70	13,16	6,69	25,95	5,44	26,07	60
	/entregador/encomendaDisponibilizada	13,15	15,18	7,46	26,07	2,43	31,09	60
	/entregador/pontoRecebimento	20,97	23,77	8,47	36,02	5,66	46,35	66
Edison	/bloco/disponivel	225,26	271,96	176,36	463,48	148,65	1254,29	50
	/colaboracao/fim	220,82	237,04	65,66	361,04	157,73	361,05	15
	/entregador/coletaDisponivel	230,53	357,35	477,02	853,43	155,01	3204,84	50
	/entregador/encomendaColetada	213,79	268,14	287,32	384,10	157,83	2219,76	50
	/entregador/encomendaDisponibilizada	235,04	283,87	182,41	572,59	150,05	1272,54	50
	/entregador/pontoRecebimento	234,50	283,57	171,32	425,30	157,51	1122,78	55
Nuvem	/bloco/disponivel	225,26	271,96	176,36	463,48	148,65	1254,29	50
	/colaboracao/fim	220,82	237,04	65,66	361,04	157,73	361,05	15
	/entregador/coletaDisponivel	230,53	357,35	477,02	853,43	155,01	3204,84	50
	/entregador/encomendaColetada	213,79	268,14	287,32	384,10	157,83	2219,76	50
	/entregador/encomendaDisponibilizada	235,04	283,87	182,41	572,59	150,05	1272,54	50
	/entregador/pontoRecebimento	234,50	283,57	171,32	425,30	157,51	1122,78	55

Fonte: elaborada pelo autor (2026).

APÊNDICE B – MÉTRICAS DE LATÊNCIA E JITTER POR TÓPICO MQTT — CENÁRIO COM INFERÊNCIA EMBARCADA

Ambiente	Tópico	Lat. Mediana (ms)	Jitter (ms)	Amostras
Local	/bloco/disponivel	10,53	6,08	20
	/cam/capture	51,10	4,49	20
	/colaboracao/fim	13,21	5,09	4
	/entregador/coletaDisponivel	15,57	5,11	20
	/entregador/encomendaColetada	10,58	5,91	20
	/entregador/encomendaDisponibilizada	5,49	3,51	20
	/entregador/pontoRecebimento	15,58	4,99	21
	/esp/classificar	320,60	93,37	20
	/esp/resultado	164,30	30,14	20
Edison	/bloco/disponivel	130,27	59,15	20
	/cam/capture	151,90	127,98	20
	/colaboracao/fim	142,75	6,53	4
	/entregador/coletaDisponivel	149,89	36,77	20
	/entregador/encomendaColetada	132,11	126,52	20
	/entregador/encomendaDisponibilizada	129,65	53,76	20
	/entregador/pontoRecebimento	160,17	61,35	21
	/esp/classificar	259,27	120,86	20
	/esp/resultado	258,88	31,59	20
AWS	/bloco/disponivel	205,86	24,77	20
	/cam/capture	442,27	35,42	20
	/colaboracao/fim	249,69	8,77	4
	/entregador/coletaDisponivel	214,17	32,23	20
	/entregador/encomendaColetada	212,99	20,24	20
	/entregador/encomendaDisponibilizada	202,32	62,29	20
	/entregador/pontoRecebimento	224,78	59,10	21
	/esp/classificar	742,79	92,08	20
	/esp/resultado	472,19	71,55	20

Fonte: elaborada pelo autor (2026).

APÊNDICE C – TEMPO DE INFERÊNCIA NO ESP32-S3 POR ARQUITETURA DE MODELO

Modelo	Ambiente	Média (ms)	Mín (ms)	Máx (ms)	Desvio (ms)	N
CNN Autoral	Local	53,80	53,154	54,087	0,22	20
CNN Autoral	Edison	53,87	53,229	54,657	0,32	20
CNN Autoral	AWS	53,78	53,234	54,133	0,26	20
MobileNetV3 Small	AWS*	118,12	116,183	121,042	1,49	32
MobileNetV2	AWS*	333,13	330,826	337,659	1,71	37

Fonte: elaborada pelo autor (2026).

APÊNDICE D – DADOS BRUTOS DE LATÊNCIA E *JITTER* — CENÁRIO SEM INFERÊNCIA EMBARCADA

A seguir é apresentada a primeira página dos dados brutos coletados no cenário sem inferência embarcada. A versão completa está disponível no repositório do projeto: <https://github.com/Keltonmd/EdgeBoxAI/blob/main/resultados/apendice_D_brutos_sem_edge.pdf>

Ambiente	Tópico	Pub.	Sub.	Lat. (ms)
Local	/bloco/disponivel	sensor	Franka	10,89
Local	/bloco/disponivel	sensor	Franka	0,64
Local	/bloco/disponivel	sensor	Franka	5,64
Local	/bloco/disponivel	sensor	Franka	10,73
Local	/bloco/disponivel	sensor	Franka	20,97
Local	/bloco/disponivel	sensor	Franka	26,10
Local	/bloco/disponivel	sensor	Franka	26,26
Local	/bloco/disponivel	sensor	Franka	10,72
Local	/bloco/disponivel	sensor	Franka	15,86
Local	/bloco/disponivel	sensor	Franka	15,84
Local	/bloco/disponivel	sensor	Franka	10,97
Local	/bloco/disponivel	sensor	Franka	10,72
Local	/bloco/disponivel	sensor	Franka	26,08
Local	/bloco/disponivel	sensor	Franka	10,82
Local	/bloco/disponivel	sensor	Franka	21,14
Local	/bloco/disponivel	sensor	Franka	3,16
Local	/bloco/disponivel	sensor	Franka	15,85
Local	/bloco/disponivel	sensor	Franka	10,79
Local	/bloco/disponivel	sensor	Franka	15,85
Local	/bloco/disponivel	sensor	Franka	10,74
Local	/bloco/disponivel	sensor	Franka	26,49
Local	/bloco/disponivel	sensor	Franka	10,76
Local	/bloco/disponivel	sensor	Franka	5,69
Local	/bloco/disponivel	sensor	Franka	20,98
Local	/bloco/disponivel	sensor	Franka	5,77
Local	/bloco/disponivel	sensor	Franka	20,91
Local	/bloco/disponivel	sensor	Franka	10,59
Local	/bloco/disponivel	sensor	Franka	10,94
Local	/bloco/disponivel	sensor	Franka	10,64
Local	/bloco/disponivel	sensor	Franka	5,59
Local	/bloco/disponivel	sensor	Franka	5,75
Local	/bloco/disponivel	sensor	Franka	5,71
Local	/bloco/disponivel	sensor	Franka	5,47
Local	/bloco/disponivel	sensor	Franka	26,27
Local	/bloco/disponivel	sensor	Franka	10,74
Local	/bloco/disponivel	sensor	Franka	26,00
Local	/bloco/disponivel	sensor	Franka	26,11
Local	/bloco/disponivel	sensor	Franka	5,68
Local	/bloco/disponivel	sensor	Franka	10,61
Local	/bloco/disponivel	sensor	Franka	10,85
Local	/bloco/disponivel	sensor	Franka	10,93
Local	/bloco/disponivel	sensor	Franka	15,93
Local	/bloco/disponivel	sensor	Franka	22,07
Local	/bloco/disponivel	sensor	Franka	21,02
Local	/bloco/disponivel	sensor	Franka	10,76

APÊNDICE E – DADOS BRUTOS DE LATÊNCIA E *JITTER* — CENÁRIO COM INFERÊNCIA EMBARCADA

A seguir é apresentada a primeira página dos dados brutos coletados no cenário com inferência embarcada (*Edge AI*). A versão completa está disponível no repositório do projeto: <https://github.com/Keltonmd/EdgeBoxAI/blob/main/resultados/apendice_E_brutos_com_edge.pdf>

Ambiente	Tópico	Pub.	Sub.	Lat. (ms)
Local	/bloco/disponivel	sensor	Franka	21,15
Local	/bloco/disponivel	sensor	Franka	1,24
Local	/bloco/disponivel	sensor	Franka	5,65
Local	/bloco/disponivel	sensor	Franka	10,88
Local	/bloco/disponivel	sensor	Franka	16,07
Local	/bloco/disponivel	sensor	Franka	21,13
Local	/bloco/disponivel	sensor	Franka	20,98
Local	/bloco/disponivel	sensor	Franka	10,55
Local	/bloco/disponivel	sensor	Franka	15,58
Local	/bloco/disponivel	sensor	Franka	10,57
Local	/bloco/disponivel	sensor	Franka	5,65
Local	/bloco/disponivel	sensor	Franka	15,57
Local	/bloco/disponivel	sensor	Franka	5,59
Local	/bloco/disponivel	sensor	Franka	5,46
Local	/bloco/disponivel	sensor	Franka	16,22
Local	/bloco/disponivel	sensor	Franka	10,50
Local	/bloco/disponivel	sensor	Franka	5,70
Local	/bloco/disponivel	sensor	Franka	5,46
Local	/bloco/disponivel	sensor	Franka	5,44
Local	/bloco/disponivel	sensor	Franka	10,50
Local	/cam/capture	UR10	Cam	51,36
Local	/cam/capture	UR10	Cam	61,18
Local	/cam/capture	UR10	Cam	55,55
Local	/cam/capture	UR10	Cam	51,62
Local	/cam/capture	UR10	Cam	53,72
Local	/cam/capture	UR10	Cam	47,41
Local	/cam/capture	UR10	Cam	55,02
Local	/cam/capture	UR10	Cam	58,53
Local	/cam/capture	UR10	Cam	54,24
Local	/cam/capture	UR10	Cam	45,78
Local	/cam/capture	UR10	Cam	46,35
Local	/cam/capture	UR10	Cam	45,55
Local	/cam/capture	UR10	Cam	47,04
Local	/cam/capture	UR10	Cam	54,72
Local	/cam/capture	UR10	Cam	47,99
Local	/cam/capture	UR10	Cam	46,41
Local	/cam/capture	UR10	Cam	52,31
Local	/cam/capture	UR10	Cam	48,13
Local	/cam/capture	UR10	Cam	50,83
Local	/cam/capture	UR10	Cam	48,54
Local	/colaboracao/fim	UR10	Franka	13,22
Local	/colaboracao/fim	UR10	Cam	13,25
Local	/colaboracao/fim	UR10	sensor	3,05
Local	/colaboracao/fim	UR10	youBot	13,21
Local	/entregador/coletaDisponivel	youBot	UR10	10,61

APÊNDICE F – DIAGRAMA DE ATIVIDADES DO SISTEMA IORT PROPOSTO

